RedAlimenta — Infraestructura Kubernetes para desarrollo y producción

Práctica integradora individual de Infraestructura de Servicios

Campo	Valor
Alumno	Neftali Odin Garcia Ramirez
Nombre del proyecto	RedAlimenta — gestión de donativos, caducidades y entregas de un banco de alimentos comunitario
Repositorio y versión de entrega	redalimenta/ · artefactos 1.0.0 y 1.1.0 (digests en k8s/overlays/*/kustomization.yaml)
Fecha de inicio y de entrega	19 oct 2026 (propuesta) · 31 oct 2026 (implementación y pruebas)
Ambientes	Desarrollo k3d-ra-dev (1 nodo) · Producción de laboratorio k3d-ra-prod (1 control plane + 2 workers)

Control de versiones del documento

Versión	Fecha	Cambio
1.0	2026-10-31	Implementación completa, pruebas E01–E14 y expediente D01–D10

RedAlimenta — Infraestructura Kubernetes para desarrollo y producción
Resumen del proyecto
D01 · Ficha del proyecto y requerimientos
Problema y usuarios
Reglas de negocio propias
Requerimientos funcionales y criterios de aceptación
Requerimientos no funcionales
D02 · Investigación, comparación de tecnologías y decisiones
Conceptos de Kubernetes aplicados al proyecto
Comparación de distribuciones Kubernetes
Opciones de entrada de tráfico
Registro de decisiones de arquitectura
D03 · Diagramas, inventario y capacidad
Arquitectura de la aplicación
Arquitectura de implementación
Arquitectura de comunicación
Inventario de máquinas
Consumo medido y capacidad disponible
D04 · Modelos de datos, consistencia y reglas
Modelo relacional (PostgreSQL)
Modelo no relacional (MongoDB)
Consistencia entre motores
Replicación y failover (diseño, ampliación no implementada)
D05 · Manual de instalación desde cero

Paso 1 · Preparar el equipo
Paso 2 · Crear los clústeres
Paso 3 · Habilitar la plataforma
Paso 4 · Controles iniciales y secretos
Paso 5 · Datos, migraciones y aplicación
Paso 6 · Publicar desarrollo y probar
Paso 7 · Promover a producción
Paso 8 · Validar y reconstruir
D06 · Seguridad
Por capa
Matriz RBAC (namespace redalimenta)
Matriz de flujos y firewall
Riesgos residuales
D07 · Publicación, promoción, actualización, rollback y accesos
D08 · Operación, observabilidad, alertas e incidentes
Manual de incidentes
D09 · Respaldos, restauración y continuidad
D10 · Plan de pruebas, resultados y conclusiones
Matriz de aceptación
Experimento de escalado horizontal (E06)
Pruebas de fallos (E11), actualización (E12) y persistencia (E05)
Capturas de la aplicación
Barreras encontradas
Limitaciones
Conclusiones
Referencias
Anexo · Índice de evidencias

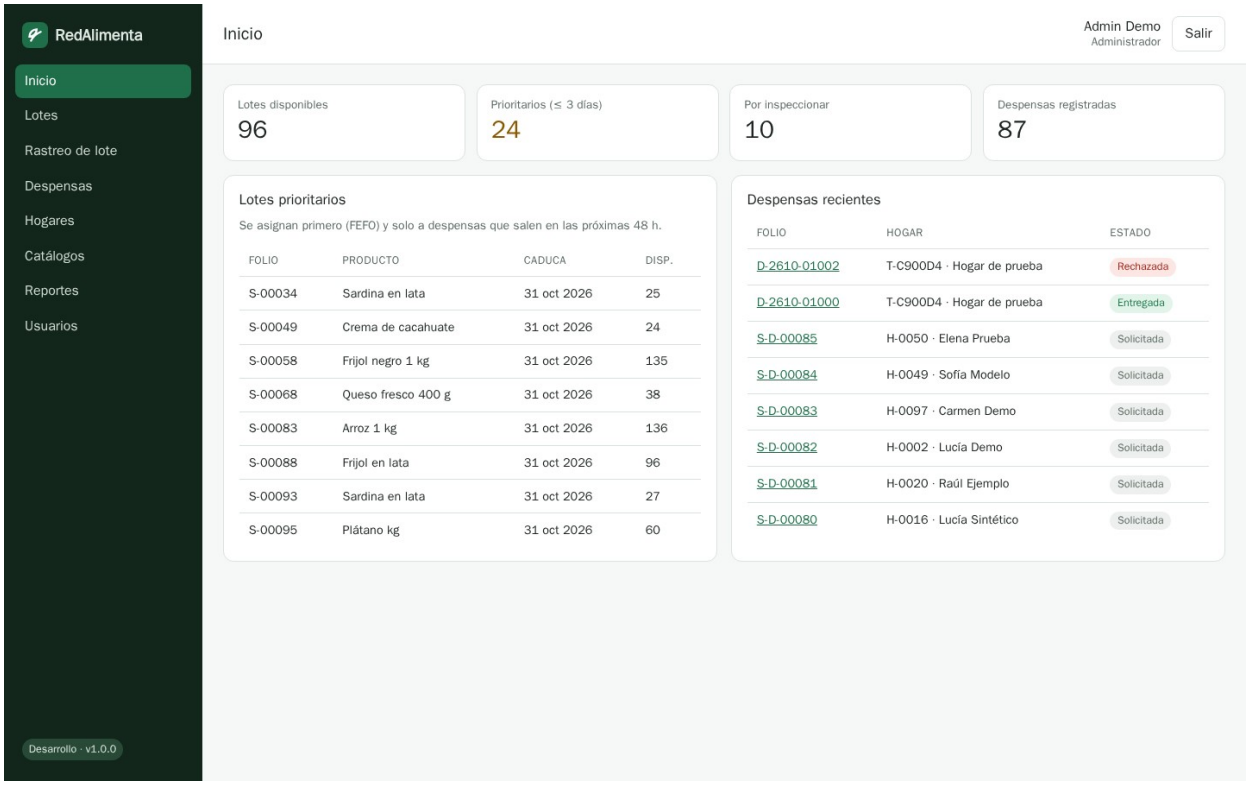
Resumen del proyecto

RedAlimenta controla cada lote de alimento donado desde que entra al almacén hasta que llega a un hogar. El sistema aplica seis reglas de negocio propias (FEFO obligatorio, estados automáticos por caducidad, una despensa cada 14 días con raciones por integrante, bloqueo de alérgenos, cadena de frío y retiro sanitario) y mantiene un expediente de trazabilidad por lote. Todos los datos son sintéticos.

La aplicación es multicapa: una SPA React servida por Nginx, una API FastAPI modular sin estado, **PostgreSQL** como fuente de verdad y **MongoDB** como expediente flexible alimentado mediante el **patrón outbox**. Se implementó en **dos clústeres k3s independientes** (creados con k3d sobre Docker): desarrollo con un nodo y producción de laboratorio con un control plane y dos workers. Cada ambiente tiene su propio contexto, sus credenciales, su CA, sus volúmenes y su URL HTTPS.

La plataforma usa Calico (CNI con NetworkPolicy), MetalLB (IP externa), Traefik con **Gateway API**, cert-manager (TLS con CA de laboratorio), local-path (PVC), Kustomize (base + overlays), kube-prometheus-stack, Loki y Alloy. En producción el backend escala con HPA de 2 a 6 réplicas distribuidas entre workers.

Resultados principales: 22/22 pruebas funcionales en ambos ambientes; HPA escaló hasta 6 réplicas y regresó a 2 (tras diagnosticar y corregir una restricción de distribución); restauración aislada verificada con **RTO medido de 78 s**; firewall de nodos y NetworkPolicy validados con controles positivos y negativos; actualización a 1.1.0 y rollback sin errores de servicio; alertas activadas y resueltas sin enviar datos a terceros.



Pantalla de inicio del administrador (desarrollo)

D01 · Ficha del proyecto y requerimientos

Problema y usuarios

Un banco de alimentos recibe donativos con caducidad corta y lleva el control en hojas de cálculo. Eso produce **merma**, porque se entrega primero lo que llegó al último; **reparto desigual**, porque un hogar recibe varias despensas y otros ninguna; y **falta de trazabilidad**, porque ante un lote contaminado no se sabe qué hogares lo recibieron.

Rol	Qué hace	No puede (verificado en servidor)
Administrador	Usuarios, catálogos, reportes, retiro sanitario	Confirmar entregas en nombre de otro
Coordinador de almacén	Recibe donativos, inspecciona lotes, arma despensas (FEFO)	Aprobar despensas, dar de alta hogares
Trabajador social	Registra hogares, solicita, aprueba o rechaza despensas	Modificar inventario
Voluntario de entrega	Ve despensas listas de su ruta y confirma la entrega	Ver hogares fuera de su ruta

Reglas de negocio propias

1. **FEFO**: al armar una despensa se asigna el lote disponible con la caducidad más próxima. El backend rechaza (409) que se elija un lote más nuevo si hay uno anterior disponible.
2. **Caducidad cambia el estado**: un lote a 3 días o menos de caducar pasa a *Prioritario* y solo va a despensas que se entregan en las siguientes 48 h. Un lote vencido pasa a *Merma*. Lo aplica una tarea diaria (CronJob).
3. **Frecuencia y tamaño**: un hogar recibe como máximo una despensa cada 14 días. Raciones = 1 + 0.5 por integrante adicional, con tope de 4. Un índice único parcial impide que un hogar tenga dos despensas abiertas.
4. **Alérgenos bloqueados**: no se asignan productos con un alérgeno registrado del hogar.
5. **Cadena de frío**: un refrigerado con más de 5 °C en la inspección queda *Rechazado*.
6. **Retiro sanitario**: el lote pasa a *Retirado*, se cancelan las despensas pendientes que lo contienen (liberando el resto del stock) y se listan los hogares que ya lo recibieron.

Requerimientos funcionales y criterios de aceptación

ID	Requerimiento	Criterio de aceptación	Evidencia
RF1	CRUD de Lote, Hogar y Despensa (+ catálogos Donante y Producto)	Alta, consulta, modificación y baja (física solo sin historia; lógica en el resto) desde UI y API	test_crud_*, capturas 04–12
RF2	Autenticación y 4 roles	401 sin sesión, 403 por rol, sesión expirada, CSRF obligatorio	test_csrf_obligatorio, test_*_no_*, test_sesion_expirada, E03-sesion-expirada-*.txt
RF3	Validación en cliente y servidor	422 por campo (tipos, rangos, obligatorios), 409 por unicidad o regla	test_validacion_*, captura 11
RF4	Búsqueda, paginación y reporte mensual	Página 2 de 10 con total ≥ 100; kilos, merma por categoría y hogares	test_paginacion_y_busqueda, captura 07
RF5	Flujo de despensa con estados	Transiciones inválidas → 409; rol incorrecto → 403	test_flujo_completo_*
RF6	Expediente en MongoDB	Pantalla *Rastreo de lote* y lista de hogares receptores	captura 06, E04
RF7	Semilla reproducible	9 usuarios, 12 donantes, 20 productos, 160 lotes, 120 hogares, 85 despensas	log del Job migraciones

Requerimientos no funcionales

ID	Requerimiento	Criterio	Resultado
RNF1	Desempeño	p95 < 1 s y errores < 1 % bajo la rampa definida (60 VU)	Cumple: p95 99.7 ms con HPA (150 ms sin HPA), 0 % de errores en 86 299 solicitudes
RNF2	Disponibilidad de la capa de aplicación	≥ 2 réplicas en workers distintos; PDB; caída de un Pod sin pérdida de servicio	E07, E11
RNF3	Seguridad	PSA Restricted, NetworkPolicy default-deny, TLS extremo a extremo y hacia las BD en prod	E08, E09
RNF4	Recuperación	RPO 24 h, RTO 30 min	RTO medido 78 s
RNF5	Reproducibilidad	Reconstrucción por scripts sin secretos en Git	E14

D02 · Investigación, comparación de tecnologías y decisiones

Conceptos de Kubernetes aplicados al proyecto

- **Plano de control y nodos.** En prod, k3d-ra-prod-server-0 corre API server, scheduler y controladores (k3s los integra en un proceso, con kine/SQLite como almacén). Tiene un taint NoSchedule, así que las cargas van a los workers agent-0 y agent-1. El kubelet de cada nodo ejecuta los contenedores con containerd.
- **Pod y Deployment.** backend, frontend y outbox son Deployments con Pods intercambiables. Las actualizaciones son graduales (maxSurge 1, maxUnavailable 0) y se conservan 10 revisiones para hacer rollback.
- **Service y DNS.** backend y frontend son ClusterIP; postgres y mongo son headless (postgres-0.postgres.redalimenta.svc). CoreDNS resuelve los nombres (10.53.0.10 en prod), de modo que la app nunca depende de IP de Pods.
- **CNI y NetworkPolicy.** Calico 3.33 con VXLAN aplica 11 políticas: denegación inicial de entrada y salida, y luego solo los flujos declarados.
- **CSI y almacenamiento.** El StorageClass local-path (provisionador de k3s) usa WaitForFirstConsumer, así que el PV queda ligado al nodo donde se programó el Pod.
- **StatefulSet.** PostgreSQL y MongoDB tienen identidad estable y un PVC por réplica (datos-postgres-0). El StatefulSet **no replica** el motor: la replicación se diseña como ampliación (D04).
- **Configuración y secretos.** ConfigMap app-config generado por Kustomize (con hash), Secrets por consumidor (backend, migraciones, respaldos, exporters) y cifrado en reposo (--secrets-encryption).
- **RBAC.** Roles lector, operador y desplegador ligados a grupos de certificados de cliente. Las ServiceAccounts de la app tienen automountServiceAccountToken: false.
- **Sondas.** El backend usa startup /healthz, readiness /readyz (PostgreSQL) y liveness /healthz (sin BD, para evitar reinicios masivos).
- **HPA.** autoscaling/v2 sobre CPU al 60 % de los requests (250m), con minReplicas 2 y maxReplicas 6.
- **Distribución de réplicas.** topologySpreadConstraints por kubernetes.io/hostname con maxSkew 1 y nodeTaintsPolicy: Honor.
- **Actualizaciones y recuperación.** Rolling update, kubectl rollout undo y migraciones compatibles hacia atrás (0002).

Comparación de distribuciones Kubernetes

Criterio	k3s vía k3d (elegida)	kubeadm en VMs	minikube
Recursos	~500 MiB por nodo; 4 nodos caben en un equipo de 30 GiB	2 GiB+ por VM y un hipervisor	Pensado para un solo clúster
Multinodo	Sí (servers + agents como contenedores)	Sí (real)	Limitado
Soporte	CNCF certificada, versiones fijadas (rancher/k3s:v1.37.1-k3s1)	Upstream oficial	Upstream, orientado a desarrollo
Seguridad	Cifrado de Secrets, auditoría y PSA por flags	Completo, con configuración manual	Básico
Reproducción	Un archivo YAML por clúster; recreación en ~5 min	Requiere VMs, sudo y libvirt	Fácil, pero un solo clúster
Limitación	Los nodos comparten kernel y host: no demuestra tolerancia a la pérdida del host	—	—

kubeadm no fue viable porque el equipo no tiene sudo ni libvirt para crear VMs.

Opciones de entrada de tráfico

Criterio	Traefik v3.7 + Gateway API (elegida)	ingress-nginx + Ingress
Modelo	Gateway/HTTPRoute con roles separados (plataforma vs aplicación)	Ingress con anotaciones del controlador
Mantenimiento	Activo; implementación conforme de Gateway API v1	El proyecto comunitario anunció su retiro (marzo 2026)

Criterio	Traefik v3.7 + Gateway API (elegida)	ingress-nginx + Ingress
Redirección HTTP→HTTPS	Filtro estándar `RequestRedirect`	Anotación propietaria
Seguridad	Se ejecuta como no root con FS de solo lectura	Equivalente

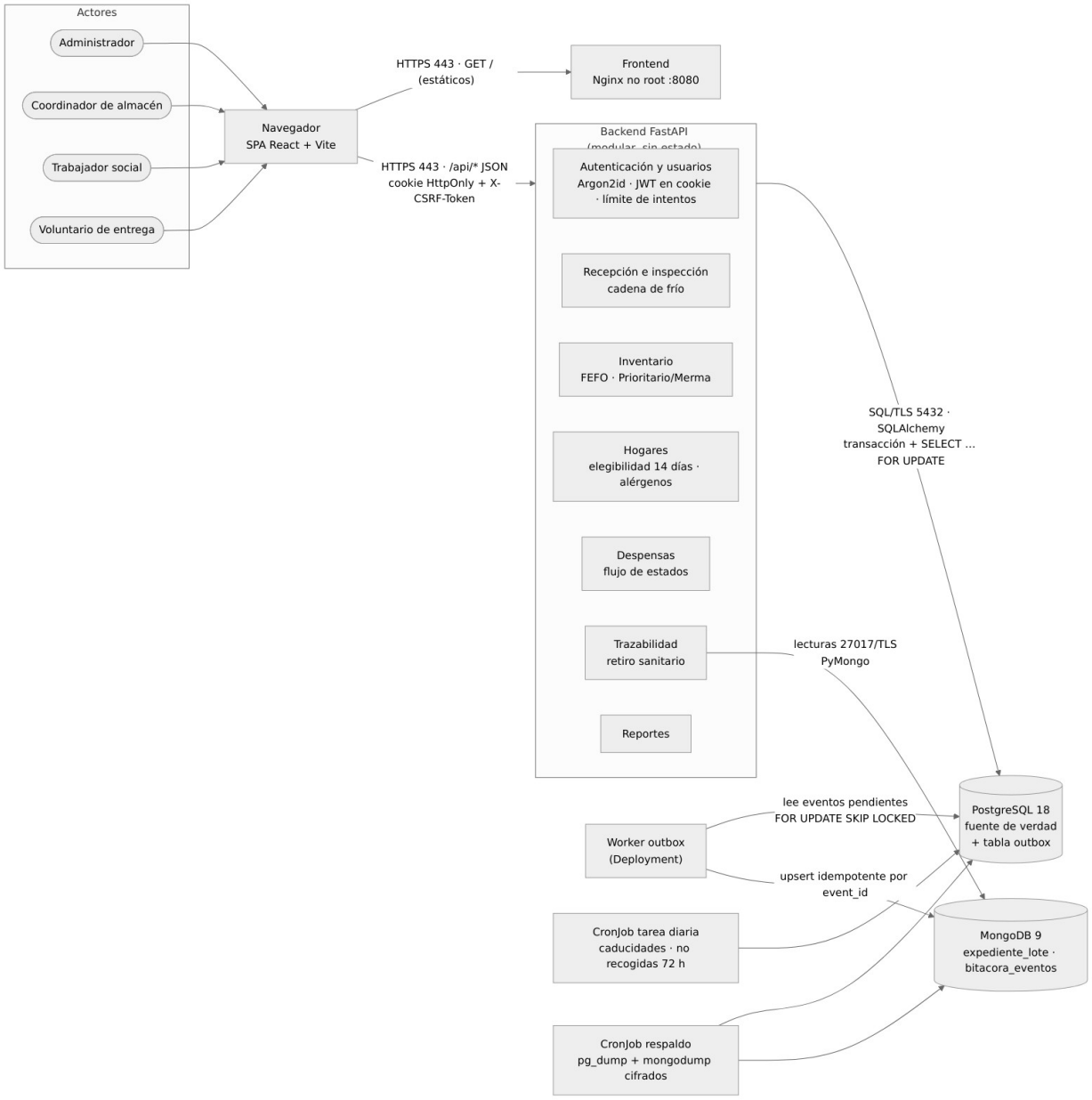
La IP externa la asigna **MetalLB 0.16 en modo L2** (pool 172.31.0.200–210 en prod, 172.30.0.200–210 en dev). Sin MetalLB, el Service `LoadBalancer` de Traefik quedaría en `Pending`.

Registro de decisiones de arquitectura

#	Decisión	Alternativa descartada	Motivo	Costo / limitación
1	k3s 1.37 vía k3d	kubeadm en VMs	Sin sudo ni hipervisor; reproducible en minutos	Mismo host físico = dominio de fallo único
2	Calico VXLAN	Flannel	Flannel no aplica NetworkPolicy	~250m de CPU por nodo; requiere `allow_ip_forwarding` en k3d
3	Traefik + Gateway API + MetalLB L2	ingress-nginx + ServiceLB (klipper)	Estándar mantenido; IP dedicada por ambiente	L2: un solo speaker responde ARP (balanceador único)
4	local-path en ambos ambientes	**Longhorn** (propuesto)	Longhorn requiere `open-iscsi` en los nodos; los nodos k3d son contenedores sin iscsiadm (barrera)	Volumen ligado al nodo: si el worker cae, la BD espera
5	PostgreSQL 18 + MongoDB 9 con outbox	Escritura dual directa	Sin atomicidad entre motores; outbox con reintento idempotente	Latencia de proyección ~1 s
6	kube-prometheus-stack + Loki + Alloy	Metrics Server solo / ELK	Historial, alertas y logs con correlación; menos memoria que ELK	~2 GiB adicionales en prod
7	Kustomize base + overlays con digests	Helm propio	Sin plantillas; el diff de promoción es solo el digest	—
8	Registro OCI local compartido	Importar imágenes a cada clúster	Promoción por digest inmutable	Punto único del laboratorio

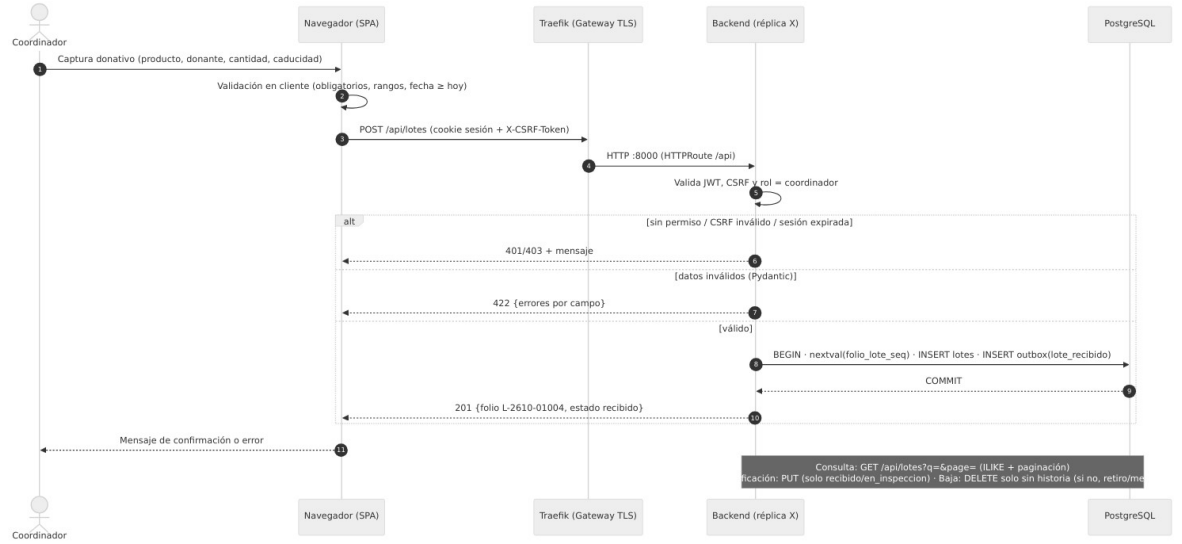
D03 · Diagramas, inventario y capacidad

Arquitectura de la aplicación

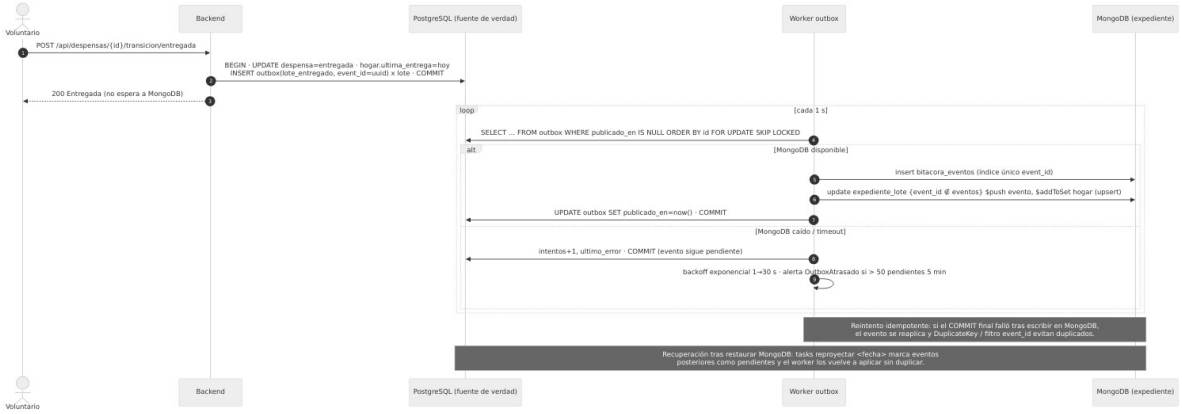


Arquitectura de la aplicación: actores, SPA, API modular, autenticación y ambos motores

Recorrido de referencia. Navegador → DNS (sslip.io resuelve a la IP de MetalLB) → firewall del nodo (la entrada web pasa por DNAT de kube-proxy) → Traefik con terminación TLS (Gateway listener `https` :8443, expuesto como 443) → HTTPRoute. `/api/*` va al Service `backend` y el resto al Service `frontend`. El backend consulta los Services internos `postgres:5432` y `mongo:27017` (con TLS en prod). La interfaz es una SPA: Nginx solo entrega archivos estáticos y el navegador llama directamente a `/api` en el mismo origen. Por eso no hace falta CORS, la cookie es `SameSite=Strict` y el navegador nunca accede a las bases de datos.

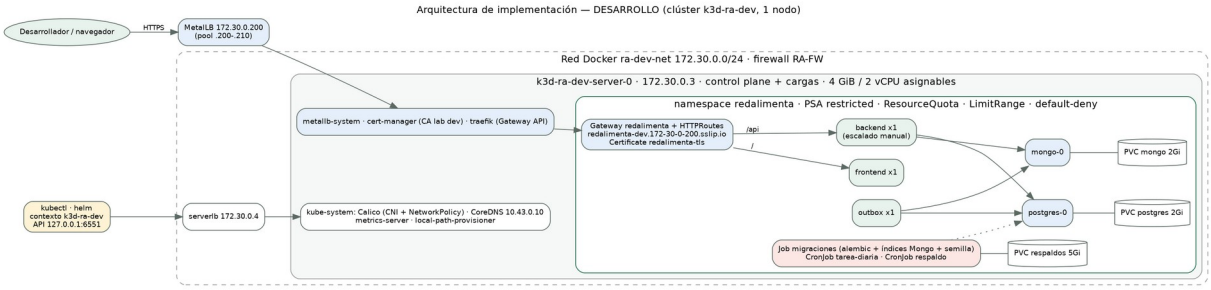


Secuencia de un CRUD (alta de lote) con validación, autorización y errores

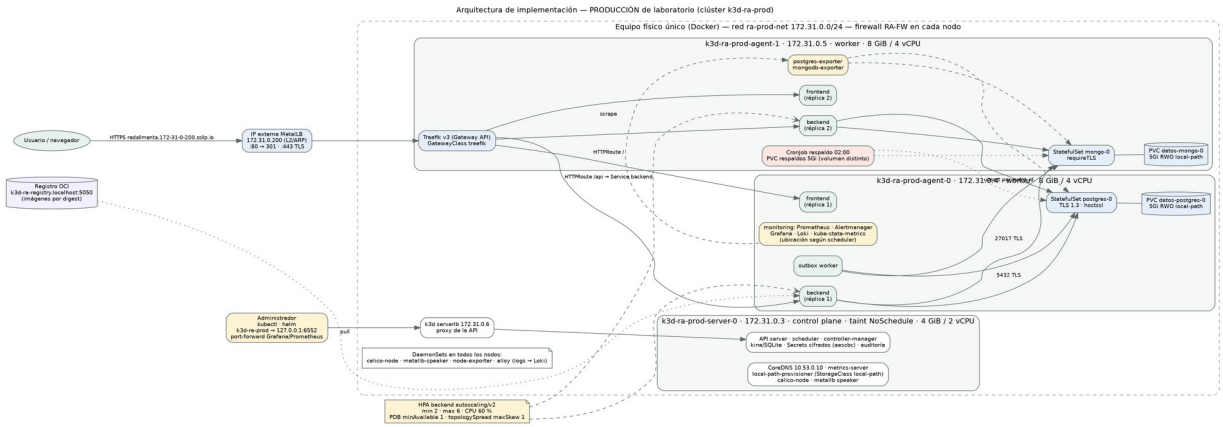


Secuencia del flujo que usa ambos motores: entrega, outbox, caída de MongoDB y recuperación

Arquitectura de implementación

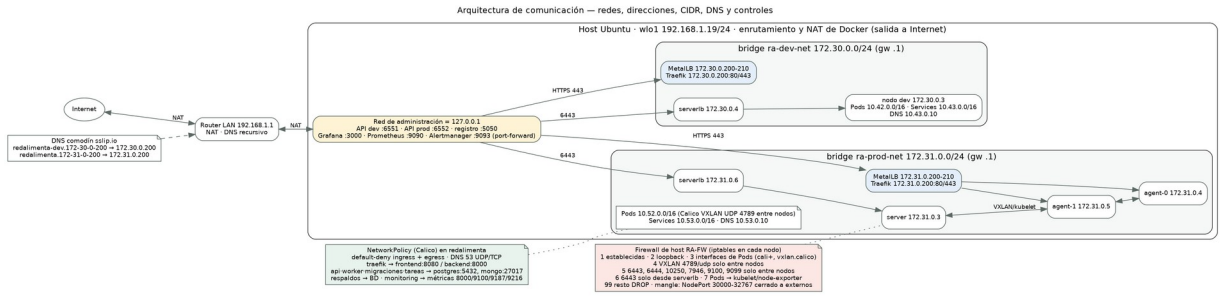


Desarrollo: un nodo con plataforma y aplicación



Producción: control plane con taint, dos workers, réplicas distribuidas, PVC locales y observabilidad

Arquitectura de comunicación



Redes, direcciones de nodos, CIDR de Pods y Services, rangos de MetalLB, DNS, firewall y políticas

Los rangos no se solapan con la LAN 192.168.1.0/24 ni con las redes Docker preexistentes (172.17/16 y 172.18/16): nodos dev 172.30.0.0/24, nodos prod 172.31.0.0/24, Pods 10.42/16 y 10.52/16, Services 10.43/16 y 10.53/16. No hay VLAN ni VPN; el NAT de salida a Internet lo hace Docker en el host. La red de administración es el loopback del equipo: las APIs están publicadas solo en 127.0.0.1.

Inventario de máquinas

Máquina	Rol	CPU	RAM	Disco	SO / arquitectura	IP
Host físico	Equipo administrador y anfitrión Docker	Intel i7-9750H, 12 hilos	30 GiB	455 GB LVM	Ubuntu 26.04.1, kernel 7.0.0-38, x86_64, Docker 29.1.3	192.168.1.19 (wlo1)
k3d-ra-dev-server-0	Control plane + cargas (dev)	2 vCPU asignables (system-reserved 10)	4 GiB (límite Docker)	compartido	K3s v1.37.1, containerd 2.3.4	172.30.0.3
k3d-ra-prod-server-0	Control plane (taint)	2 vCPU asignables	4 GiB	compartido	K3s v1.37.1	172.31.0.3
k3d-ra-prod-agent-0	Worker	4 vCPU asignables (system-reserved 8)	8 GiB	compartido	K3s v1.37.1	172.31.0.4
k3d-ra-prod-agent-1	Worker	4 vCPU asignables	8 GiB	compartido	K3s v1.37.1	172.31.0.5

Docker asigna las IP de los nodos dentro del /24 al crearlos; la tabla refleja la reconstrucción final (E14). Las pruebas de E08 se hicieron con la asignación anterior (agent-1 .4, agent-0 .5). El firewall calcula las IP en tiempo de ejecución y no depende de esta asignación.

El presupuesto inicial (2 vCPU y 4 GiB para dev y para el control plane; 4 vCPU y 8 GiB por worker) se aplicó con serversMemory, agentsMemory y kubelet system-reserved, de modo que el scheduler vea exactamente esa capacidad. La CPU física es compartida (12 hilos para todo).

Consumo medido y capacidad disponible

Medición en reposo después de las pruebas (kubectl top, evidencia D03-consumo-medido.txt), comparada con el presupuesto:

Nodo	CPU usada	Memoria usada	Requests asignados (CPU / memoria)	Presupuesto
prod server-0	199m	1.4 GiB (33 %)	270m / 32 Mi	2 vCPU / 4 GiB
prod agent-0	390m	3.2 GiB (38 %)	1560m (39 %) / 2.0 Gi	4 vCPU / 8 GiB
prod agent-1	291m	2.5 GiB (30 %)	980m (24 %) / 1.3 Gi	4 vCPU / 8 GiB
dev server-0	324m	2.2 GiB (54 %)	—	2 vCPU / 4 GiB

Consumo por componente en prod (reposo): Prometheus 1.07 GiB, Grafana 514 MiB, Loki 257 MiB, MongoDB 117 MiB, backend ~78 MiB por réplica, outbox 55 MiB, PostgreSQL 49 MiB, Calico ~65 MiB por nodo y Alloy ~70 MiB por nodo. **La observabilidad consume más memoria que la aplicación.** Bajo carga máxima (6 réplicas): backend 2.52 cores y 523 MiB en total, PostgreSQL 1.03 cores, y la CPU de los nodos al ~52 %. Ajuste: el presupuesto inicial alcanza; dev queda al 54 % de memoria, así que no conviene agregar observabilidad completa en dev sin subirlo a 6 GiB.

Un clúster multinodo de contenedores sobre un solo host permite practicar programación, distribución y drain, pero **no** demuestra tolerancia a la pérdida del host. Los puntos únicos de fallo identificados son: el control plane único (kine/SQLite), el speaker L2 de MetalLB que anuncia la IP, una sola instancia de cada base sin réplica, PVC locales ligados a un nodo y el registro OCI local. El HPA crea Pods, no máquinas: con 6 réplicas de backend (1.5 vCPU en requests) cada worker conserva capacidad, y si se pidieran más réplicas de las que caben, quedarían en Pending y se activaría la alerta PodsPendientes.

D04 · Modelos de datos, consistencia y reglas

Modelo relacional (PostgreSQL)

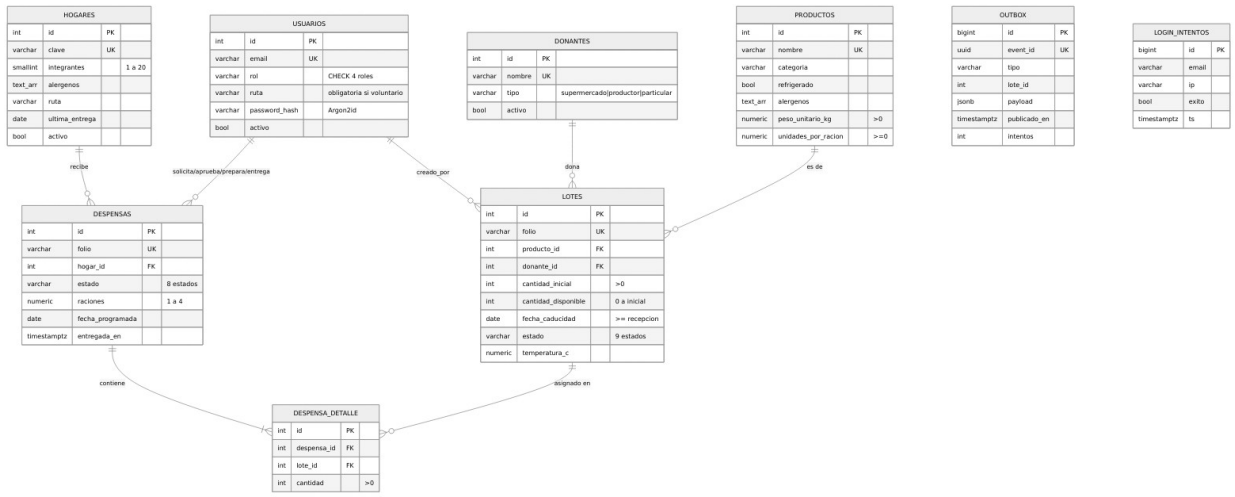


Diagrama entidad-relación

Tabla	Llave y restricciones principales	Índices
usuarios	PK id · email UNIQUE · CHECK rol ∈ 4 roles · CHECK voluntario ⇒ ruta	UNIQUE email
donantes	PK · nombre UNIQUE · CHECK tipo	—
productos	PK · nombre UNIQUE · CHECK categoría · peso > 0 · unidades_por_racion ≥ 0 · alergenos text[]	—
lotes	PK · folio UNIQUE (secuencia) · FK producto, donante, usuario · cantidad_inicial > 0 · 0 ≤ disponible ≤ inicial · caducidad ≥ recepción · CHECK estado (9)	(estado, fecha_caducidad); parcial FEFO (producto_id, fecha_caducidad) WHERE asignable
hogares	PK · clave UNIQUE · integrantes 1..20 · alergenos text[]	(id, ultima_entrega); ruta
despensas	PK · folio UNIQUE · FK hogar y usuarios · raciones 1..4 · CHECK estado (8)	(hogar_id, estado); **UNIQUE parcial hogar_id WHERE abierta**; entregada_en
despensa_detalle	PK · FK despensa (CASCADE) y lote · cantidad > 0 · UNIQUE(despensa, lote)	lote_id
outbox	PK bigserial · event_id uuid UNIQUE · payload jsonb · publicado_en	parcial pendientes; creado_en (0002)
login_intentos	PK · email, ip, éxito, ts	(email, ts)

Transacciones. El armado FEFO ocurre en una sola transacción con `SELECT … FOR UPDATE OF lotes`, ordenado por caducidad. Dos réplicas del backend no pueden asignar la misma existencia; los bloqueos siguen el orden de producto para evitar interbloqueos. Los folios usan secuencias (`folio_lote_seq`, `folio_despensa_seq`), que son seguras con varias réplicas.

Migraciones. Alembic (`backend/migrations/versions/0001`, `0002`) corre una sola vez en el Job `migraciones` con la cuenta `redalimenta_migrator` y un bloqueo consultivo (`pg_advisory_lock`). Nunca corre en cada réplica.

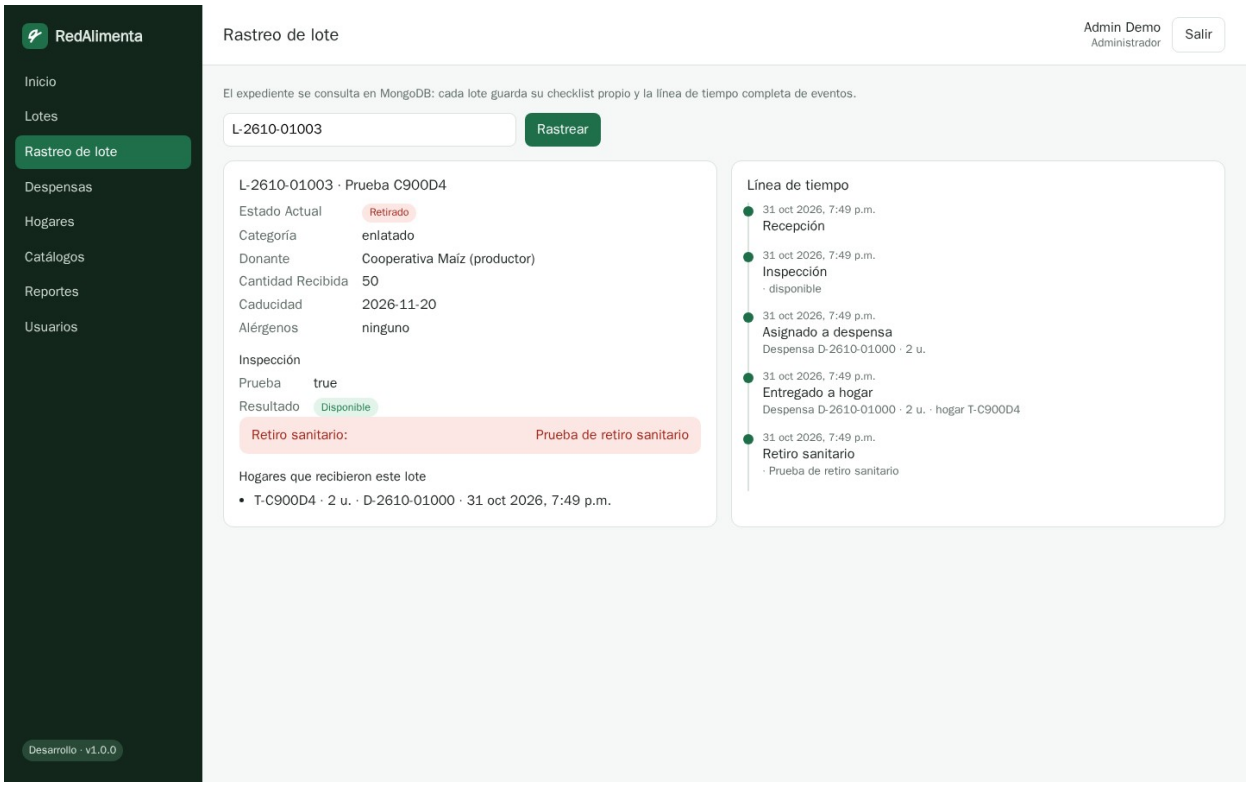
Estrategia de conexiones. Cada réplica usa un pool de SQLAlchemy de 5 + 5 overflow con `pre_ping`. Con 6 réplicas son 60 conexiones, más el outbox, por debajo de `max_connections=100` y del `CONNECTION LIMIT 80` del usuario de la app.

Modelo no relacional (MongoDB)

Colección	Documento	Validación	Índices
expediente	Uno por lote: producto, donante,	`$jsonSchema` strict (folio, lote_id,	ux_folio_lote (único),

Colección	Documento	Validación	Índices
_lote	inspeccion con **checklist flexible por categoría**, eventos[] (línea de tiempo), hogares_receptores[], retiro	eventos con event_id/tipo/ts)	eventos.hogar_id, eventos.event_id
bitacora_eventos	Un documento por evento	$jsonSchema strict	ux_event_id (único), (hogar_id, ts), (folio_lote, tipo), (tipo, ts)

Valor de la base no relacional. La inspección de un refrigerado guarda lecturas de temperatura, la de un enlatado guarda el lote del fabricante y el estado de la lata, y la de fruta guarda el grado de madurez. Cada expediente es un documento con su propio checklist y un arreglo de eventos que crece sin esquema fijo. La pantalla *Rastreo de lote* y la consulta "¿qué hogares recibieron el lote X?" dependen de MongoDB (índice ix_folio_tipo, 6 documentos examinados para 6 devueltos; evidencia E04).



Expediente de MongoDB con retiro sanitario y hogares a avisar

E04 · Traza PostgreSQL → MongoDB

```
# E04 Uso funcional y trazable de ambos motores (dev) 2026-10-31T20:48:33Z
== PostgreSQL: último evento confirmado en la tabla outbox (misma transacción que el cambio de negocio)
 id  |              event_id               |      tipo      |    folio     |  hogar   |         publicado_en
-----+-------------------------------------+----------------+--------------+----------+-------------------------------
 865 | e0d7ed69-4282-48e8-9813-e853b5dd68c4 | lote_entregado | L-2610-01009 | T-68A780 | 2026-10-31 20:42:39.665407+00
(1 row)

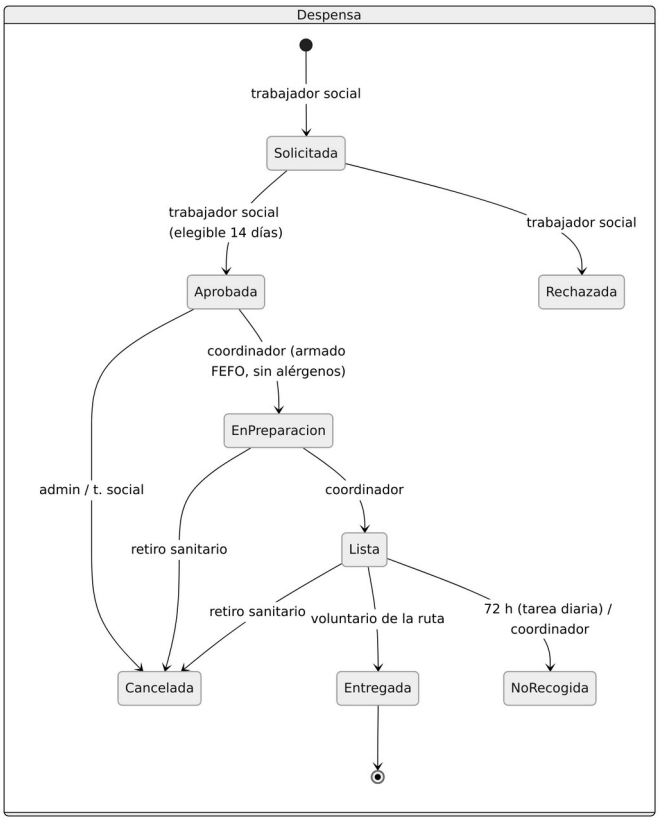
== MongoDB: el mismo event_id en bitacora_eventos y dentro del expediente del lote
{
  ts: '2026-10-31T20:42:39.592078+00:00',
  tipo: 'lote_entregado',
  lote_id: 170,
  cantidad: 2,
  despensa: 'D-2610-01003',
  event_id: 'e0d7ed69-4282-48e8-9813-e853b5dd68c4',
  hogar_id: 128,
  folio_lote: 'L-2610-01009',
  usuario_id: 6,
  hogar_clave: 'T-68A780'
}
{
  folio_lote: 'L-2610-01009',
  estado: 'retirado',
  hogares_receptores: [
    {
      hogar_id: 128,
      clave: 'T-68A780',
      despensa: 'D-2610-01003'
    }
  ]
}
== Consulta '¿qué hogares recibieron el lote?' usa índice (explain)
plan: "ix_folio_tipo"  docsExaminados: 6  devueltos: 6
== Consulta FEFO en PostgreSQL usa el índice parcial ix_lotes_producto_fefo
SET
                                                                QUERY PLAN

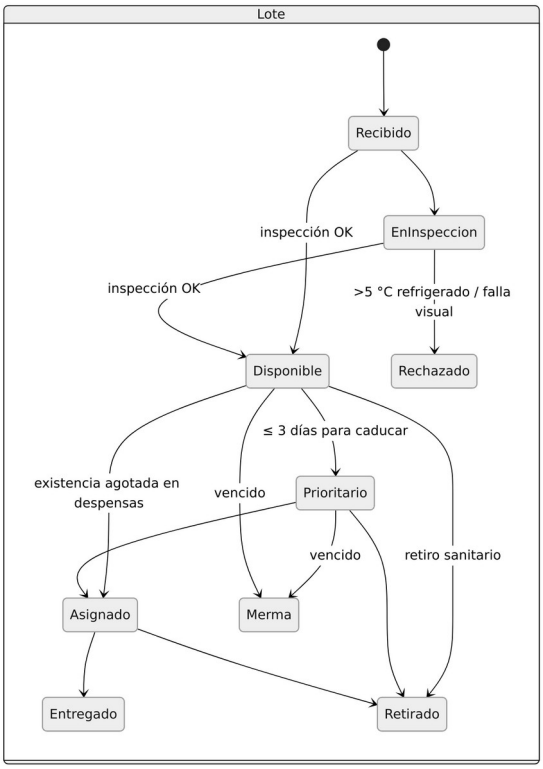
------------------------------------------------------------------------------------------------------------------------------------------
----------------------
 LockRows  (cost=11.72..11.78 rows=5 width=111)
   ->  Sort  (cost=11.72..11.73 rows=5 width=111)
         Sort Key: fecha_caducidad, id
         ->  Bitmap Heap Scan on lotes  (cost=4.21..11.66 rows=5 width=111)
               Recheck Cond: ((producto_id = 1) AND (fecha_caducidad >= CURRENT_DATE) AND ((estado)::text = ANY
('{disponible,prioritario}'::text[])))
== Validación JSON Schema de MongoDB rechaza documentos inválidos
rechazado: Document failed validation
== Índice único event_id impide duplicados
rechazado:
```

Traza del mismo event_id desde la tabla outbox hasta MongoDB, uso de índices y validación de esquema

Consistencia entre motores

- **Fuente de verdad:** PostgreSQL. MongoDB es una proyección histórica.
- Cada cambio de estado escribe en la misma transacción el cambio de negocio y una fila en `outbox` con un `event_id` UUID. No se supone atomicidad entre motores.
- El worker `outbox` toma eventos pendientes en orden (`FOR UPDATE SKIP LOCKED`), los inserta en `bitacora_eventos` (índice único event_id) y los agrega al expediente solo si su `event_id` no está ya en `eventos`. Si el COMMIT final falla después de escribir en MongoDB, el reintento no duplica nada.
- Si MongoDB está caído, los eventos esperan con `intentos` y `ultimo_error`, el worker reintenta con backoff exponencial (1→30 s) y la operación del almacén sigue (E11).
- Tras restaurar MongoDB desde un respaldo, `python -m app.tasks reproyectar <fecha>` vuelve a marcar como pendientes los eventos posteriores y el worker los reaplica sin duplicar.





Estados de la despensa y del lote

Replicación y failover (diseño, ampliación no implementada)

- **PostgreSQL:** operador CloudNativePG con 1 primario y 2 réplicas en streaming, en workers distintos (antiafinidad) y con PVC replicado. La elección de primario la hace el operador mediante lease en la API de Kubernetes; la réplica sincrónica (`synchronous_commit=remote_apply`) evita perder transacciones confirmadas; el fencing del primario anterior evita tener dos primarios. Las conexiones van por el Service `-rw`.

- **MongoDB:** replica set de 3 miembros (o 2 + árbitro) con `w: majority` y `readConcern majority`. La elección por mayoría impide el split-brain, y la URL incluye `replicaSet=rs0` para que el driver siga al primario.
- Ninguna de las dos replicaciones reemplaza a los respaldos verificados (D09).

D05 · Manual de instalación desde cero

Cada paso indica objetivo, ambiente, prerrequisitos, archivo, comando exacto (sin secretos), resultado esperado, evidencia y diagnóstico. Las versiones están fijadas en infra/platform/versions.env.

Paso 1 · Preparar el equipo

- **Objetivo:** tener las herramientas sin privilegios y los límites del host. **Ambiente:** equipo administrador. **Prerrequisitos:** Docker 29 sin sudo (grupo docker), python3 y acceso a Internet.
- **Archivo:** infra/scripts/00-prerequisitos.sh.

```
infra/scripts/00-prerequisitos.sh
sudo sysctl -w fs.inotify.max_user_instances=1024 fs.inotify.max_user_watches=524288   # recomendado
```

- **Resultado esperado:** kubectl v1.37.2, k3d v5.9.0, helm v4.3.1 y trivy 0.76.1 en ~/.local/bin.
- **Aplicado en este equipo:** /etc/sysctl.d/99-kubernetes-k3d.conf con 1024 instancias y 524288 watches.
- **Error posible:** con max_user_instances=128, los Pods que usan inotify fallan con "too many open files" (ocurrió con el frr-k8s-statuscleaner de MetalLB). Solución: subir el límite, o desactivar FRR si solo se usa L2 (metallb-values.yaml).

Paso 2 · Crear los clústeres

- **Objetivo:** tener dos clústeres independientes con sus topologías. **Archivos:** infra/clusters/k3d-dev.yaml, k3d-prod.yaml, audit-policy.yaml, infra/platform/calico/render.py.

```
infra/scripts/01-clusters.sh
kubectl config get-contexts
kubectl --context k3d-ra-prod get nodes -o wide
```

- **Parámetros:** --flannel-backend=none y --disable-network-policy (para usar Calico), --disable=traefik,servicelb (Traefik se instala con Helm y la IP externa la da MetalLB), CIDR propios, --secrets-encryption, política de auditoría, system-reserved para simular la capacidad, taint del control plane y API en 127.0.0.1:6551/6552.
- **Resultado esperado:** 1 nodo Ready en dev y 3 en prod (E01). Duración medida: **4 min 39 s** para ambos.
- **Errores y diagnóstico:** k3d rechaza subnet en una red con nombre ("cannot specify subnet for existing network"), por eso el script crea las redes antes. Si los nodos quedan NotReady, revisar las sondas de calico-node: el manifiesto se renderiza sin -bird-live/-bird-ready porque VXLAN no usa BIRD.
- **Reversión:** k3d cluster delete ra-dev ra-prod.

kubectl config get-contexts · get nodes (dev y prod)

```
# E01 2026-10-31T20:47:56Z
== Host
Architecture:                         x86_64
CPU(s):                               12
Model name:                           Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
CPU(s) scaling MHz:                   76%
               total        used        free      shared  buff/cache   available
Mem:            30Gi        12Gi       886Mi       731Mi        18Gi        18Gi
/dev/mapper/ubuntu--vg-ubuntu--lv  455G   63G  370G  15% /
PRETTY_NAME="Ubuntu 26.04.1 LTS"
7.0.0-38-generic
Docker 29.1.3
wlo1 192.168.1.19/24
== Contextos
CURRENT   NAME          CLUSTER       AUTHINFO            NAMESPACE
*         k3d-ra-dev    k3d-ra-dev    admin@k3d-ra-dev
          k3d-ra-prod   k3d-ra-prod   admin@k3d-ra-prod
== k3d-ra-dev nodos
NAME                  STATUS   ROLES           AGE   VERSION        INTERNAL-IP   EXTERNAL-IP   OS-IMAGE           KERNEL-
VERSION            CONTAINER-RUNTIME
k3d-ra-dev-server-0   Ready    control-plane   78m   v1.37.1+k3s1   172.30.0.3    <none>        K3s v1.37.1+k3s1
7.0.0-38-generic (amd64)   containerd://2.3.4-k3s1
NODO                  CPU_ASIGNABLE   MEM_ASIGNABLE   TAINTS
k3d-ra-dev-server-0   2               4294967Ki       <none>
== k3d-ra-dev componentes del sistema
cert-manager    cert-manager-574945dccf-jqj5r            1/1  Running    k3d-ra-dev-server-0
cert-manager    cert-manager-cainjector-65f5d8fdc9-nkct6  1/1  Running    k3d-ra-dev-server-0
cert-manager    cert-manager-webhook-58b9c7b6cc-wszxj    1/1  Running    k3d-ra-dev-server-0
kube-system     calico-kube-controllers-846669895c-wzh7h  1/1  Running    k3d-ra-dev-server-0
kube-system     calico-node-gq7mg                        1/1  Running    k3d-ra-dev-server-0
kube-system     coredns-577d995dff-r6s2b                 1/1  Running    k3d-ra-dev-server-0
kube-system     helm-install-gateway-api-crd-p9qw6       0/1  Completed  k3d-ra-dev-server-0
kube-system     local-path-provisioner-6858d854cf-tm4lc  1/1  Running    k3d-ra-dev-server-0
kube-system     metrics-server-778c4649b4-dt4ns          1/1  Running    k3d-ra-dev-server-0
metallb-system  metallb-controller-ffcf647c4-4rbx6       1/1  Running    k3d-ra-dev-server-0
metallb-system  metallb-speaker-xf8jr                    1/1  Running    k3d-ra-dev-server-0
traefik         traefik-65cb4859dd-hd9bl                 1/1  Running    k3d-ra-dev-server-0
== k3d-ra-prod nodos
NAME                   STATUS   ROLES           AGE   VERSION        INTERNAL-IP   EXTERNAL-IP   OS-IMAGE           KERNEL-
VERSION            CONTAINER-RUNTIME
k3d-ra-prod-agent-0    Ready    <none>          76m   v1.37.1+k3s1   172.31.0.5    <none>        K3s v1.37.1+k3s1
7.0.0-38-generic (amd64)   containerd://2.3.4-k3s1
k3d-ra-prod-agent-1    Ready    <none>          76m   v1.37.1+k3s1   172.31.0.4    <none>        K3s v1.37.1+k3s1
7.0.0-38-generic (amd64)   containerd://2.3.4-k3s1
k3d-ra-prod-server-0   Ready    control-plane   76m   v1.37.1+k3s1   172.31.0.3    <none>        K3s v1.37.1+k3s1
… (34 líneas más en E01-contextos-nodos.txt)
```

Contextos y nodos de ambos clústeres

Paso 3 · Habilitar la plataforma

```
infra/scripts/02-plataforma.sh dev
infra/scripts/02-plataforma.sh prod
kubectl --context k3d-ra-prod -n traefik get svc traefik      # EXTERNAL-IP 172.31.0.200
```

- **Instala:** MetalLB 0.16.1 (pool y L2Advertisement), cert-manager v1.21.3 (CA de laboratorio por ambiente con ClusterIssuer redalimenta-ca) y Traefik chart 42.2.0 (v3.7.16) con el proveedor Gateway API y la GatewayClass traefik. Los CRD de Gateway API v1 los instala k3s. metrics-server ya viene en k3s.
- **Errores encontrados:** el chart de Traefik 42 cambió logs.access por accessLog (error de esquema). La primera instalación de MetalLB activó FRR (BGP), que no se necesita.

Paso 4 · Controles iniciales y secretos

```
infra/scripts/03-secretos.sh dev      # genera secrets/dev.env (600) si no existe y crea los Secrets
infra/scripts/03-secretos.sh prod
```

- Los namespaces, la cuota, el LimitRange, el RBAC y las NetworkPolicy se aplican con el overlay (paso 5).
- Los valores se generan con secrets.token_hex y nunca se imprimen. La CA pública se publica como ConfigMap ca-laboratorio.

Paso 5 · Datos, migraciones y aplicación

```
infra/scripts/construir.sh 1.0.0                # build + Trivy + push -> docs/evidencias/ci/build-*.txt
infra/scripts/promover.sh dev docs/evidencias/ci/build-1.0.0-202610311941.txt
infra/scripts/04-desplegar.sh dev
```

- **Resultado esperado:** postgres-0 y mongo-0 Ready; el Job migraciones termina con "Semilla aplicada: usuarios 9, donantes 12, productos 20, lotes 160, hogares 120, despensas 85, outbox 788"; backend, outbox y frontend Ready.
- **Errores encontrados y corregidos:**
  - La clave de cuota services.loadbalancer no es válida; debe ser services.loadbalancers.
  - El Job agotó su backoff mientras las BD descargaban imágenes; se agregó una espera activa.
  - Los recursos propios del overlay de prod cayeron en default porque faltaba namespace: en el overlay.

Paso 6 · Publicar desarrollo y probar

```
curl --cacert secrets/ca-dev.crt https://redalimenta-dev.172-30-0-200.sslip.io/api/version
docker run --rm --network host -v $PWD/secrets/ca-dev.crt:/ca.crt:ro -e BASE_URL=https://redalimenta-dev.172-30-0-200.sslip.io \
  -e CA_FILE=/ca.crt -e SEED_PASSWORD=<de secrets/dev.env> redalimenta-tests:dev
```

- **Resultado:** HTTP 80 → 301; HTTPS válido con la CA de dev; **22 passed**.

Paso 7 · Promover a producción

```
infra/scripts/05-observabilidad.sh prod                    # CRDs de ServiceMonitor/PrometheusRule antes del overlay
infra/scripts/promover.sh prod docs/evidencias/ci/build-1.0.0-202610311941.txt   # mismo digest aprobado en dev
infra/scripts/04-desplegar.sh prod
infra/firewall/firewall-nodos.sh prod && infra/firewall/firewall-nodos.sh dev
```

- **Resultado:** 2 réplicas de backend y frontend en workers distintos, HPA, PDB, TLS hacia las BD (TLSv1.3) y exporters; **22 passed** en prod.

Paso 8 · Validar y reconstruir

Las pruebas E01–E14 se describen en D10. Se reconstruyeron **ambos** ambientes desde cero solo con los scripts: dev en 7 min 1 s y prod en 12 min 3 s. Diferencias encontradas en la primera instalación y ya incorporadas a los scripts: redes Docker creadas antes que k3d, sondas de Calico sin BIRD, accessLog en Traefik 42, MetalLB sin FRR, clave services.loadbalancers, namespace en los overlays, espera en el Job de migraciones, nodeTaintsPolicy: Honor, Traefik con 2 réplicas y PDB, preStop en backend y frontend. Pasos que dependen del administrador: subir el límite de inotify (sudo) y conservar la copia de secrets/<amb>.env.

D06 · Seguridad

Por capa

Capa	Implementación	Evidencia
Aplicación	Argon2id; autorización por rol en cada endpoint; ORM y consultas parametrizadas; errores genéricos (detalle solo en logs); máximo 5 intentos fallidos en 15 min (en PostgreSQL, compartido entre réplicas)	E03, test_*
Navegador y API	TLS de la CA de laboratorio; cookie HttpOnly, Secure, SameSite=Strict; CSRF de doble envío (X-CSRF-Token); sin CORS (mismo origen); cabeceras CSP, HSTS, X-Frame-Options y nosniff en Nginx	test_csrf_obligatorio
Datos	Cuentas separadas: redalimenta_migrator (DDL), redalimenta_app (solo DML, sin DELETE en outbox), redalimenta_backup (solo lectura), redalimenta_exporter (pg_monitor); en Mongo, app readWrite, migrator dbAdmin, backup backup, exporter clusterMonitor	E09
Contenedores	Usuario no root, allowPrivilegeEscalation: false, capabilities drop ALL, seccomp RuntimeDefault, FS de solo lectura en app, BD y Traefik	PSA Restricted aceptó todos los Pods
Kubernetes	RBAC por función (grupos redalimenta:lectores, operadores, desplegadores); una SA por componente sin token; PSA restricted (enforce, warn, audit) en redalimenta; excepciones documentadas en metallb-system y monitoring (hostNetwork/hostPath)	E09
Secretos	Fuera de Git e imágenes (secrets/ en .gitignore); generados por script; cifrado en reposo con aescbc (k3s); auditoría de lectura de Secrets (nivel Metadata)	ver abajo
Suministro	Dependencias fijadas (requirements.txt, package-lock.json); imágenes base con versión; despliegue **por digest**; Trivy bloquea HIGH/CRITICAL corregibles	docs/evidencias/ci/
Infraestructura	API solo en 127.0.0.1; firewall RA-FW por nodo; kubeconfig y llaves con permisos 600; auditoría del API server	E08

Tratamiento de hallazgos de Trivy. El primer build fue **rechazado**. En el backend, msgpack (HIGH), setuptools y urllib3 venían de pip vendorizado en la imagen base; como pip no se usa en ejecución, se eliminó. En el frontend, brotli-libs (CRITICAL) y curl (HIGH) de Alpine se trataron con apk upgrade y eliminando curl. El segundo build pasó sin hallazgos corregibles (reportes en docs/evidencias/ci/trivy-*).

Cifrado de Secrets en reposo y TLS hacia las BD. k3s cifra los Secrets con aescbc (--secrets-encryption; verificable con k3s secrets-encrypt status, evidencia SEC-01). En prod, PostgreSQL exige hostssl (las conexiones sin TLS se rechazan con reject) y los clientes usan sslmode=verify-full con la CA del ambiente. MongoDB usa requireTLS y tlsCAFile: un cliente sin TLS recibe error. Los respaldos se cifran con AES-256-CBC (PBKDF2, 200 000 iteraciones) antes de escribirse.

Creación, rotación y recuperación de secretos (sin revelar valores). Para crearlos se usa 03-secretos.sh. La rotación está automatizada en infra/scripts/rotar-secreto.sh: genera un valor nuevo, guarda una copia del anterior (600), actualiza los Secrets, aplica ALTER ROLE y reinicia los consumidores de forma gradual. **Se ejecutó en vivo en prod** sobre redalimenta_app (SEC-04): 34 s, **0 errores en 147 solicitudes** de la sonda, la contraseña anterior queda RECHAZADA y readyz muestra PostgreSQL true. Para MongoDB el procedimiento equivalente usa db.changeUserPassword. Para recuperar, el archivo secrets/<amb>.env se resguarda cifrado fuera del equipo (con la llave de respaldos): sin él, un respaldo cifrado no se puede abrir.

Rotación de secreto en vivo (prod)

```
# Rotación de secreto en vivo (prod) 2026-10-31T22:31:46Z
huella anterior: dfc9f946992b
rotación completada: huella nueva 98a9f8978d1a
duración: 34 s
== Impacto (sonda 0.25 s):
solicitudes=147 correctas=147 fallidas=0 (0.00%) p95_ok=57 ms
== La contraseña anterior ya no autentica:
RECHAZADA: connection failed: connection to server at "10.52.3.30", port 5432 failed: FATAL:  password authentication fai
== La app funciona con el nuevo valor: HTTP 200 · readyz backend: {"postgres":true,"mongo":true}
```

Rotación de la contraseña de la cuenta de la app en vivo, sin errores

Sesión expirada (prod)

```
# Sesión expirada (prod) 2026-10-31T22:31:13Z
token vencido hace 1 min  -> {"detail":"Sesión expirada"} HTTP 401
token vigente (control)   -> HTTP 200
duración configurada de la sesión: 30 min (cookie Max-Age y claim exp)
cabecera Set-Cookie del login (atributos):
set-cookie: ra_session=<oculto>; HttpOnly; Max-Age=1800; Path=/; SameSite=strict; Secure
set-cookie: ra_csrf=<oculto>; Max-Age=1800; Path=/; SameSite=strict; Secure
```

Sesión expirada: token vencido rechazado; cookie HttpOnly, Secure y SameSite=Strict

Matriz RBAC (namespace redalimenta)

Acción	lector	operador	desplegador	SA backend
get/list Pods, Deployments, eventos y logs	sí	sí	sí	no
get Secrets	**no**	**no**	**no**	**no**
patch/scale Deployments, rollout undo	no	sí	sí	no
delete Pods, create Jobs (respaldo manual)	no	sí	sí (Jobs)	no
create pods/exec	no	no	no	no
aplicar manifiestos de la app	no	no	sí	no
recursos de otros namespaces / clúster	no	no	no	no

E09 · RBAC con identidades reales y permisos de BD

```
# 2026-10-31T20:21:33Z
== Identidad: ana.operadora grupos=["redalimenta:operadores","system:authenticated"]
  get pods                          yes
  list deployments                  yes
  get secrets                       no
  patch deployments                 yes
  delete pods                       yes
  create pods/exec                  no
  delete statefulsets               no
  create jobs                       yes
  get pods (ns kube-system)         no
  create clusterrolebindings        no
== Identidad: luis.lector grupos=["redalimenta:lectores","system:authenticated"]
  get pods                          yes
  list deployments                  yes
  get secrets                       no
  patch deployments                 no
  delete pods                       no
  create pods/exec                  no
  delete statefulsets               no
  create jobs                       no
  get pods (ns kube-system)         no
  create clusterrolebindings        no
== Acción real denegada (lector intenta leer un Secret)
  Error from server (Forbidden): secrets "backend-secrets" is forbidden: User "luis.lector" cannot get resource "secrets" in API
 group "" in the namespace "redalimenta"
== Acción real permitida (operadora consulta Pods)
  backend-674bd556db-gl68t   1/1   Running   0     20m
  backend-674bd556db-zl4vq   1/1   Running   0     20m
== ServiceAccounts de la app sin token montado
  backend-674bd556db-gl68t automount=false
  backend-674bd556db-zl4vq automount=false
  ls: cannot access '/var/run/secrets/kubernetes.io/serviceaccount': No such file or directory
  can-i (sa backend) list secrets: no
== PostgreSQL: usuario de ejecución sin DDL ni privilegios administrativos
  redalimenta_app|f|f|f
  redalimenta_backup|f|f|f
  redalimenta_exporter|f|f|f
  redalimenta_migrator|f|f|f
  app: CREATE TABLE intrusa(x int)              -> ERROR:  permission denied for schema public
  app: DROP TABLE lotes                         -> ERROR:  must be owner of table lotes
  app: DELETE FROM outbox                       -> ERROR:  permission denied for table outbox
  app: ALTER TABLE lotes ADD COLUMN x int       -> ERROR:  must be owner of table lotes
  app: SELECT count(*) FROM lotes               -> 165
  backup: INSERT INTO donantes(nombre,tipo) VALUES ('x','particular') -> ERROR:  permission denied for table donantes
  backup: SELECT count(*) FROM hogares          -> 123
  exporter: SELECT * FROM usuarios              -> ERROR:  permission denied for table usuarios
== MongoDB: usuario de la app limitado a readWrite en redalimenta
  app: db.getSiblingDB('redalimenta').expediente_lote.countDocuments() -> 166
  app: db.getSiblingDB('admin').createUser({user:'x',pwd:'y',roles:[]}) -> MongoServerError: not authorized on admin to execute
command { createUser: "x", pwd: "xxx", roles: [], lsid: {
  app: db.getSiblingDB('redalimenta').dropDatabase()                -> MongoServerError: not authorized on redalimenta to
execute command { dropDatabase: 1, lsid: { id: UUID("9144c7
  app: db.getSiblingDB('admin').system.users.find().toArray()       -> MongoServerError: not authorized on admin to execute
command { find: "system.users", filter: {}, lsid: { id: U
```

RBAC con identidades reales (certificados de cliente) y permisos mínimos de BD

Pod Security Admission (restricted)

```
# SEC-02 Pod Security Admission (restricted) en prod 2026-10-31T21:09:48Z
pod-security.kubernetes.io/audit=restricted
pod-security.kubernetes.io/enforce-version=latest
pod-security.kubernetes.io/enforce=restricted
pod-security.kubernetes.io/warn=restricted
== Intento de Pod privilegiado como root (debe rechazarse)
Error from server (Forbidden): pods "privilegiado" is forbidden: violates PodSecurity "restricted:latest": privileged (container
 "c" must no
t set securityContext.privileged=true), allowPrivilegeEscalation != false (container "c" must set
securityContext.allowPrivilegeEscalation=f
alse), unrestricted capabilities (container "c" must set securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod
or container
"c" must set securityContext.runAsNonRoot=true), runAsUser=0 (container "c" must not set runAsUser=0), seccompProfile (pod or
container "c"
must set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")
== Pod sin securityContext (también rechazado)
Error from server (Forbidden): pods "sinctx" is forbidden: violates PodSecurity "restricted:latest": allowPrivilegeEscalation !=
 false (cont
ainer "sinctx" must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "sinctx" must set
… (4 líneas más en SEC-02-psa.txt)
```

Pod Security Admission rechaza Pods privilegiados o sin securityContext

Cifrado de Secrets en reposo y auditoría

```
# SEC-01 Cifrado de Secrets en reposo (prod) 2026-10-31T20:51:58Z
time="2026-10-31T20:51:58Z" level=warning msg="Cluster CA certificate is not trusted by the host CA bundle, but the token does
not include a CA hash. Use the full token from the server's node-token file to enable Cluster CA validation."
Encryption Status: Enabled
Current Rotation Stage: start
Server Encryption Hashes: All hashes match

Active  Key Type  Name
------  --------  ----
 *      AES-CBC   aescbckey

== Valor almacenado en kine/SQLite para /registry/secrets/redalimenta/backend-secrets (primeros bytes)
     75 k8s:enc:aescbc:v1:aescbckey
== ¿Aparece el texto plano 'postgresql+psycopg://redalimenta_app' en la base del clúster?
0
== Auditoría: últimas lecturas de Secrets registradas (metadatos, sin contenido)
2026-10-31T20:51:27.368691Z system:serviceaccount:monitoring:loki watch monitoring None nivel=Metadata
2026-10-31T20:52:27.378463Z system:serviceaccount:monitoring:loki watch monitoring None nivel=Metadata
```

Secrets cifrados en reposo (aescbc) y auditoría de accesos

Matriz de flujos y firewall

Origen	Destino	Protocolo y puerto	Control	Prueba
Usuario	IP MetalLB (Traefik)	TCP 443 (80 solo redirige con 301)	Gateway + HTTPRoute	E02
Administrador	API Kubernetes	TCP 6443 solo vía serverlb (127.0.0.1:6551/6552)	RA-FW regla 06	E08
Administrador	Nodos	docker exec (no hay SSH en los nodos k3d)	Grupo docker del host	—
Nodos ↔ nodos	kubelet 10250, API 6443/6444, VXLAN 4789/udp, memberlist 7946, node-exporter 9100	Solo IP de nodos	RA-FW reglas 04–05	E08
backend/outbox/ migraciones/tareas	postgres:5432, mongo:27017	NetworkPolicy app-salida-a-datos + *-entrada	E08	
respaldos	postgres, mongo	respaldos-salida	E13	
Prometheus (monitoring)	backend 8000, outbox 9100, exporters 9187/9216	backend-entrada, metricas-entrada	targets up	
Todos los Pods	CoreDNS 53 UDP/TCP	permitir-dns	E08	
Cualquier externo	NodePort 30000–32767	DROP (mangle RA-FW-PRE)	E08	
Pods	BD por NodePort o IP pública	Prohibido por la cuota (services.nodeports: 0, loadbalancers: 0)	quota	

Las reglas exportadas están en infra/firewall/export-iptables-prod-server.txt. Orden de las reglas: 01 establecidas → 02 loopback → 03 interfaces de Pods → 04 VXLAN entre nodos → 05 plano de control entre nodos → 06 API desde serverlb → 07 Pods hacia los servicios del nodo → 08 ICMP → 99 DROP.

E08 · NetworkPolicy: permitidas y denegadas (prod)

```
# 2026-10-31T20:19:28Z ambiente=prod
== Control: servicios sanos
127.0.0.1:5432 - accepting connections
mongo      10.52.147.23
postgres   10.52.3.19
== Pod autorizado (backend-674bd556db-gl68t, rol=api)
backend -> postgres 5432: PERMITIDA
backend -> mongo 27017: PERMITIDA
backend -> Internet 1.1.1.1:443 (egress no declarado): BLOQUEADA TimeoutError
== Pod NO autorizado (intruso, sin rol)
DNS desde intruso: 10.52.3.19 (resuelve => DNS permitido)
intruso -> postgres:5432: BLOQUEADA
intruso -> mongo:27017:   BLOQUEADA
intruso -> backend:8000 (ingress):  BLOQUEADA
== Mismo Pod con la etiqueta autorizada rol=respaldos (demuestra que el bloqueo es la política)
intruso[rol=respaldos] -> postgres:5432: PERMITIDA
intruso[rol=respaldos] -> backend:8000:  BLOQUEADA  (respaldos no tiene salida al backend)
== Desde otro namespace (default) hacia la BD
default/intruso-ns -> postgres.redalimenta:5432: BLOQUEADA
== Políticas vigentes
NAME                 POD-SELECTOR                                                  AGE
app-salida-a-datos   rol in (api,migraciones,tareas,worker)                        19m
backend-entrada      app.kubernetes.io/name=backend                                19m
denegar-todo         <none>                                                        19m
exporters-salida     app.kubernetes.io/name in (mongodb-exporter,postgres-exporter)   19m
frontend-entrada     app.kubernetes.io/name=frontend                               19m
metricas-entrada     app.kubernetes.io/name in (mongodb-exporter,outbox,postgres-exporter)   19m
mongo-entrada        app.kubernetes.io/name=mongo                                  19m
permitir-dns         <none>                                                        19m
postgres-entrada     app.kubernetes.io/name=postgres                               19m
pruebas-salida       rol=pruebas                                                   19m
respaldos-salida     rol=respaldos                                                 19m
```

NetworkPolicy: conexiones permitidas y denegadas, con DNS y servicios sanos como control

E08 · Firewall de nodos desde origen no autorizado

```
== Origen no autorizado: contenedor 172.31.0.50 en ra-prod-net
API server directo 172.31.0.3:6443: SIN RESPUESTA (bloqueado)
kubelet 172.31.0.4:10250: SIN RESPUESTA (bloqueado)
NodePort Traefik 172.31.0.4:31298: SIN RESPUESTA (bloqueado)
node-exporter 172.31.0.4:9100: SIN RESPUESTA (bloqueado)
== Host administrador (172.31.0.1)
API directo al nodo 172.31.0.3:6443 (no permitido): SIN RESPUESTA (bloqueado/timeout)
NodePort 172.31.0.4:31298 (no permitido): SIN RESPUESTA (bloqueado/timeout)
API vía red de administración 127.0.0.1:6552 (permitido): RESPONDE
Entrada web HTTPS 172.31.0.200:443 (permitido): RESPONDE
== Contadores de la regla 'descartar resto' por nodo
k3d-ra-prod-server-0: 8 paquetes descartados
k3d-ra-prod-agent-0: 0 paquetes descartados
k3d-ra-prod-agent-1: 8 paquetes descartados
```

Firewall de nodos probado desde un origen no autorizado y desde el host

¿Cómo se demuestra que una conexión bloqueada falla por la política y no por un servicio caído? En la misma prueba: (1) pg_isready dentro de postgres-0 y los EndpointSlices muestran el servicio sano; (2) un Pod autorizado conecta al mismo destino al mismo tiempo; (3) DNS resuelve desde el Pod bloqueado; (4) al **mismo Pod** se le agrega la etiqueta autorizada rol=respaldos y pasa a PERMITIDA sin tocar el servicio.

Qué protege cada control. El firewall protege los puertos del nodo (incluidos los Pods con hostNetwork, como node-exporter y los speakers de MetalLB, que NetworkPolicy no cubre). NetworkPolicy protege el tráfico Pod a Pod. El tráfico que kube-proxy redirige por DNAT (entrada web) pasa por FORWARD y lo filtran las políticas del destino.

Riesgos residuales

Riesgo	Impacto	Mitigación	Responsable
Un solo host físico	Pérdida total del servicio	Respaldos extraídos fuera del clúster; migrar a VMs o nodos físicos	Alumno / operador
Control plane único (kine)	Sin cambios en el clúster mientras cae (las cargas siguen)	Respaldo de /var/lib/rancher/k3s/server; HA con 3 servers etcd (ampliación)	Operador
PVC local-path	La BD no se mueve si cae su worker (comprobado en E11)	CSI replicado (Longhorn/Rook) en nodos reales	Operador
CA de laboratorio no pública	Advertencia del navegador si no se importa la CA	Importar secrets/ca-*.crt; en producción real, ACME	Alumno
Las reglas RA-FW no persisten al reiniciar un nodo k3d	Ventana sin firewall de host	infra/firewall/asegurar-firewall.sh detecta y reaplica (probado quitando las reglas de agent-1); timer de systemd de usuario provisto en	Operador

Riesgo	Impacto	Mitigación	Responsable
		infra/firewall/systemd/ (activación manual por el administrador)	
Contraseña de MongoDB en la línea de comando de la prueba E09	Exposición local temporal dentro del contenedor	Solo durante la prueba; rotar tras la evaluación	Alumno

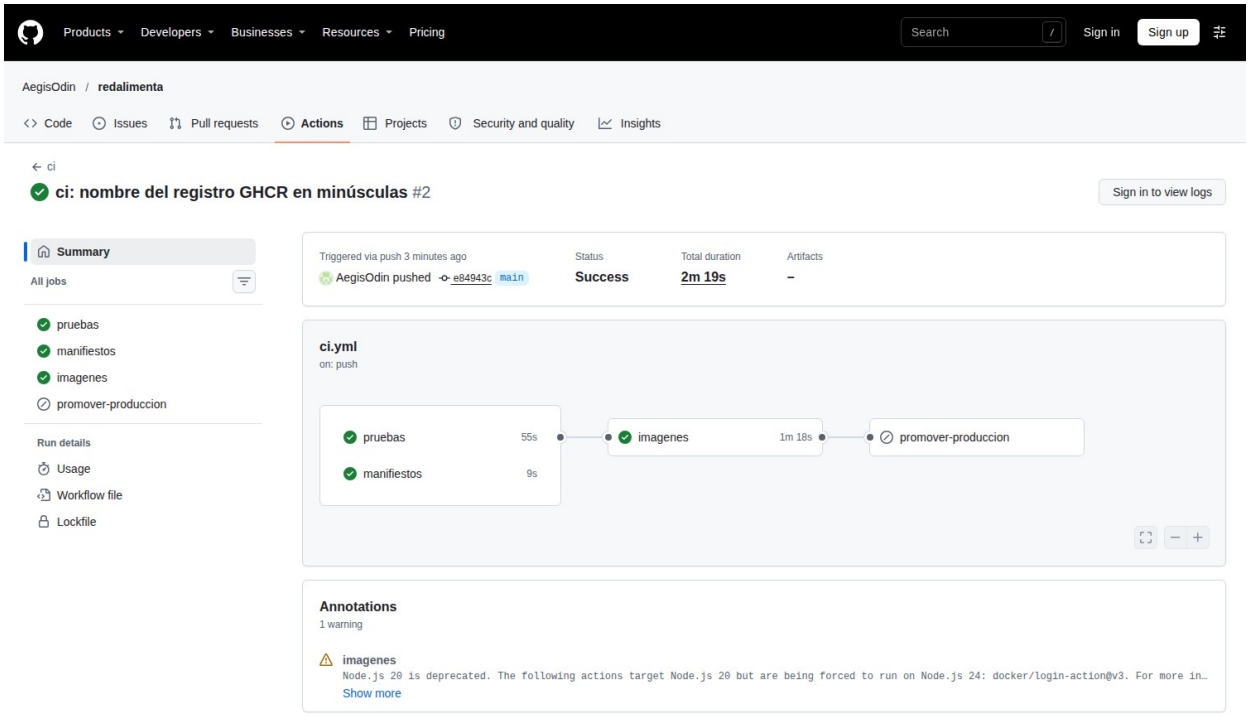
D07 · Publicación, promoción, actualización, rollback y accesos

Flujo de entrega.

7. construir.sh <versión>: build con etiqueta inmutable <versión>-<fecha>, Trivy (bloqueante), push y registro de digests en docs/evidencias/ci/build-*.txt.
8. promover.sh dev <build> + 04-desplegar.sh dev + pruebas funcionales: candidato aprobado.
9. promover.sh prod <build>: el **mismo digest** se escribe en el overlay de prod; solo cambia la configuración (réplicas, HPA, TLS, hostname).
10. 04-desplegar.sh prod: rolling update.

En GitHub Actions (.github/workflows/ci.yml) la promoción a prod requiere la aprobación del environment produccion.

Ejecución real en GitHub Actions. El repositorio está publicado en https://github.com/AegisOdin/redalimenta. La primera ejecución falló en el job imagenes porque GHCR exige nombres de repositorio en minúsculas y el propietario es AegisOdin. Se corrigió convirtiendo el nombre del registro a minúsculas dentro del paso (commit e84943c). La segunda ejecución pasó completa: pruebas (22 aprobadas, 1 omitida), manifiestos (kubeconform) e imagenes (build, Trivy sin hallazgos HIGH/CRITICAL corregibles y push a GHCR por digest). El job promover-produccion solo corre con etiquetas v* y queda detenido hasta que un revisor apruebe el environment produccion. Evidencia: docs/evidencias/ci/github-actions.txt.

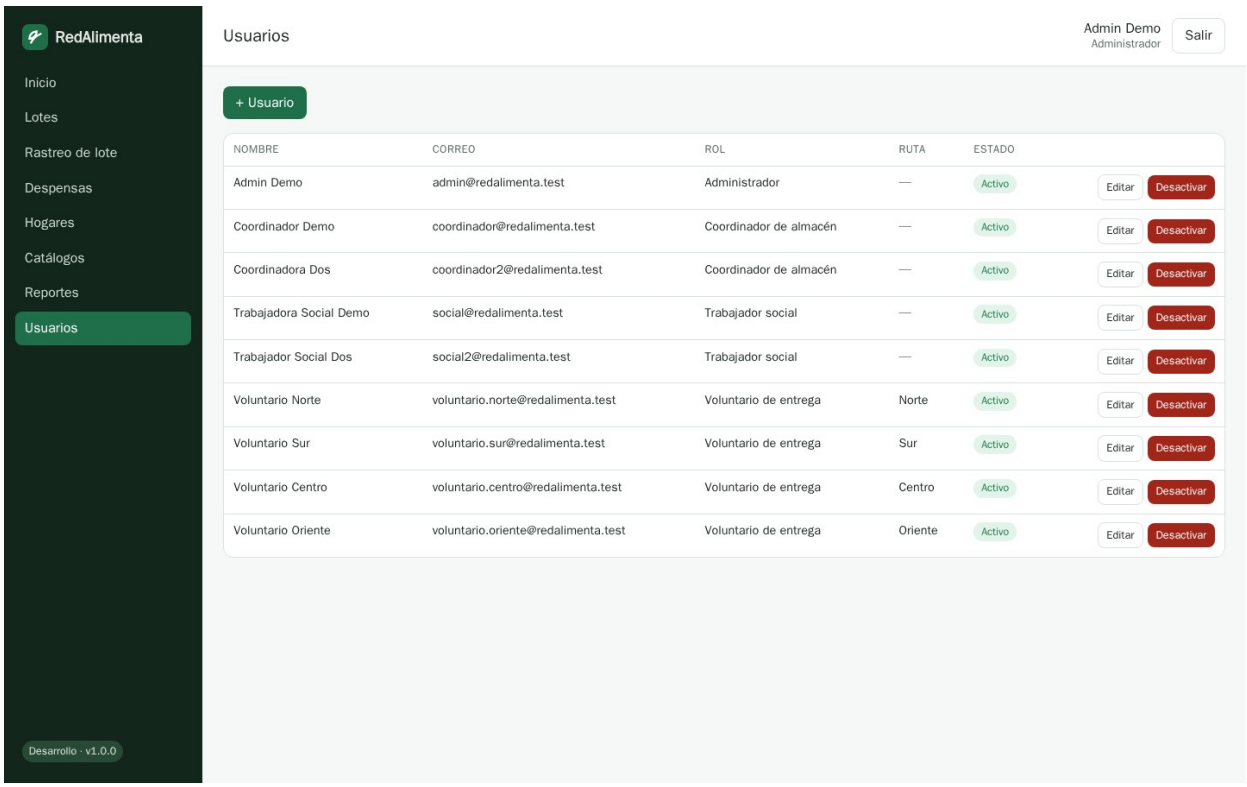


Ejecución exitosa del pipeline en GitHub Actions

Actualización y rollback (E12). La 1.1.0 incluye la migración 0002, compatible hacia atrás porque solo agrega el índice ix_outbox_creado_en. Por eso kubectl rollout undo a 1.0.0 no requiere revertir el esquema. Plan de reversión de esquema si algún día fuera incompatible: alembic downgrade 0001 con la cuenta de migración, antes del rollback de imagen y en una ventana de mantenimiento.

Administración de accesos.

- Alta: infra/scripts/usuario-k8s.sh <amb> <usuario> <grupo> (CSR API, vigencia de 30 días, kubeconfig con permisos 600).
- Baja: el certificado expira. Para revocar antes, se elimina el usuario del grupo cambiando el RoleBinding (k3s no tiene CRL de clientes).
- Usuarios de la aplicación: gestión por el administrador en la pantalla Usuarios (baja lógica).

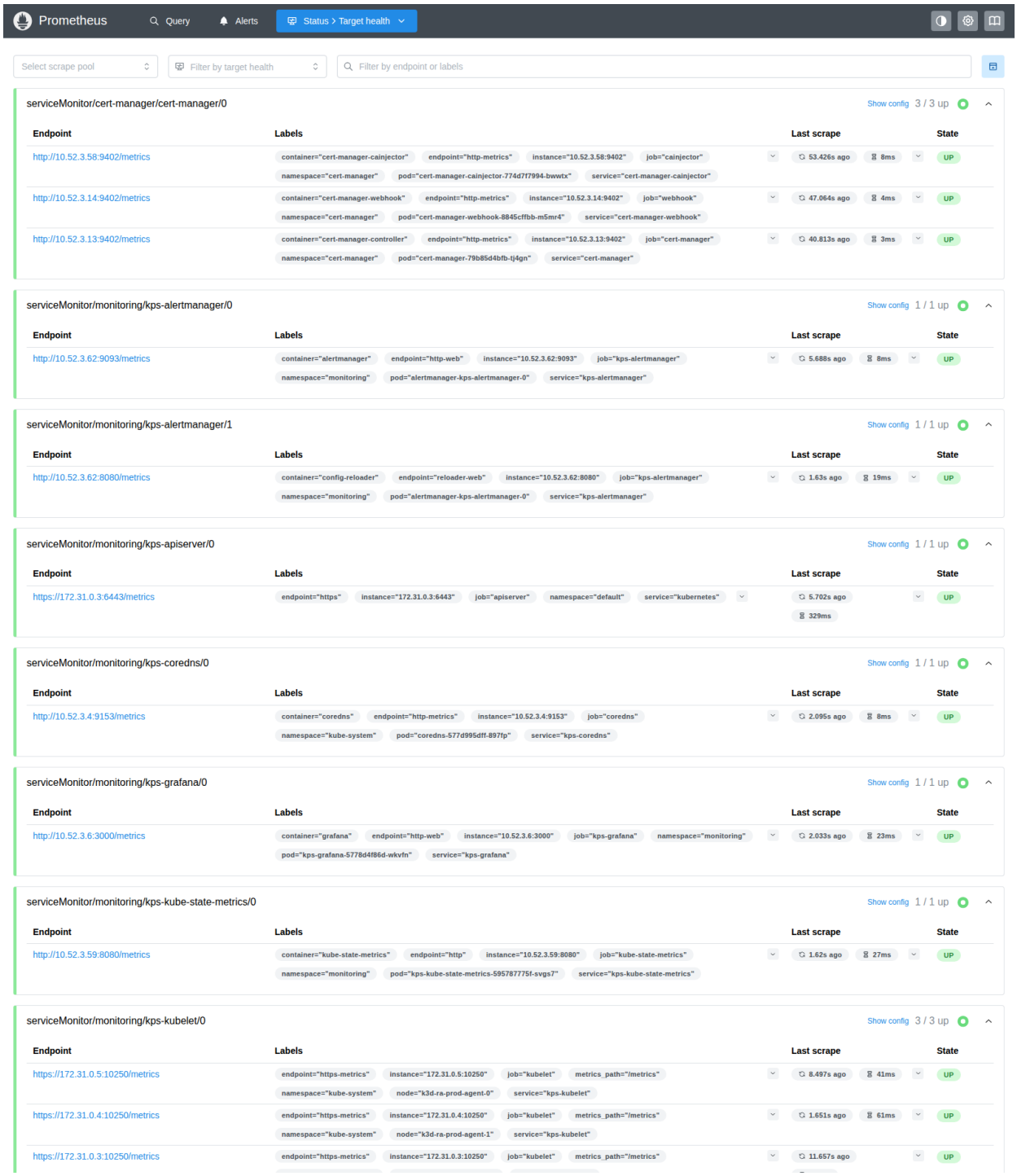


Gestión de usuarios de la aplicación (rol administrador)

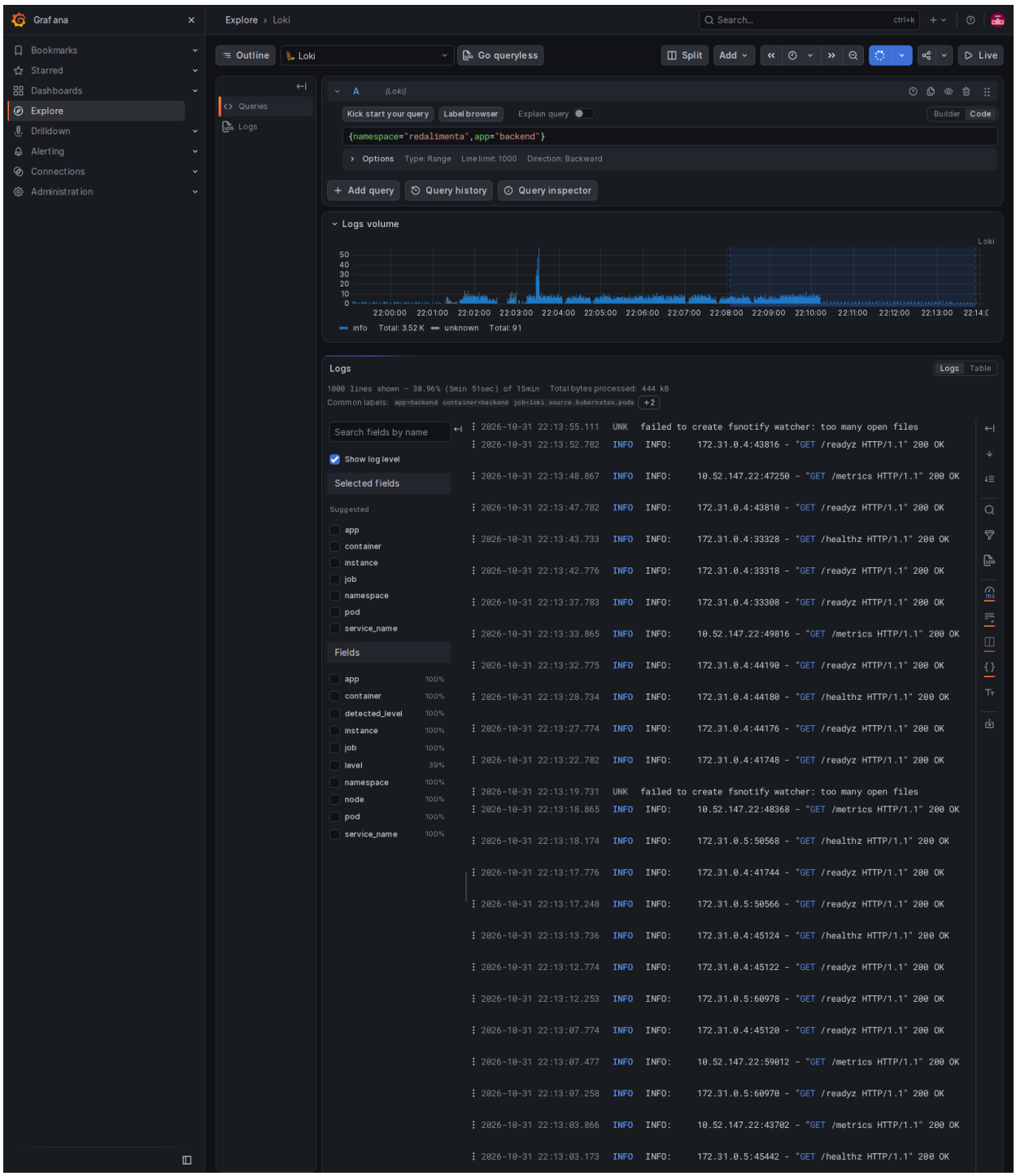
D08 · Operación, observabilidad, alertas e incidentes

Pila. Prometheus (retención de 7 días, PVC de 10 Gi), Alertmanager (receptor local, sin envío a terceros), Grafana (contraseña en Secret), node-exporter, kube-state-metrics, postgres-exporter v0.20.1, mongodb_exporter 0.54.0, Loki 3.6 (SingleBinary) y Alloy (DaemonSet). Los logs de la app son JSON con request_id de correlación (cabecera X-Request-ID). Grafana y Prometheus solo se acceden por port-forward desde la red de administración.

El panel **"RedAlimenta - Operación"** (observabilidad/dashboards/generar.py, ConfigMap detectado por el sidecar) cubre la aplicación (RPS por estado, p95/p50, tasa de 5xx, réplicas del HPA, CPU y memoria por Pod), ambos motores (disponibilidad, conexiones, transacciones, tamaño, operaciones y latencia de MongoDB, outbox), la infraestructura (CPU y memoria por nodo), los respaldos (antigüedad del último válido), los Pods no listos, las alertas activas y los logs de Loki. La captura con la carga del HPA está en D10.



Prometheus: todos los objetivos de recolección en estado up (incluye exporters de PostgreSQL y MongoDB, kubelet y node-exporter con el firewall activo)



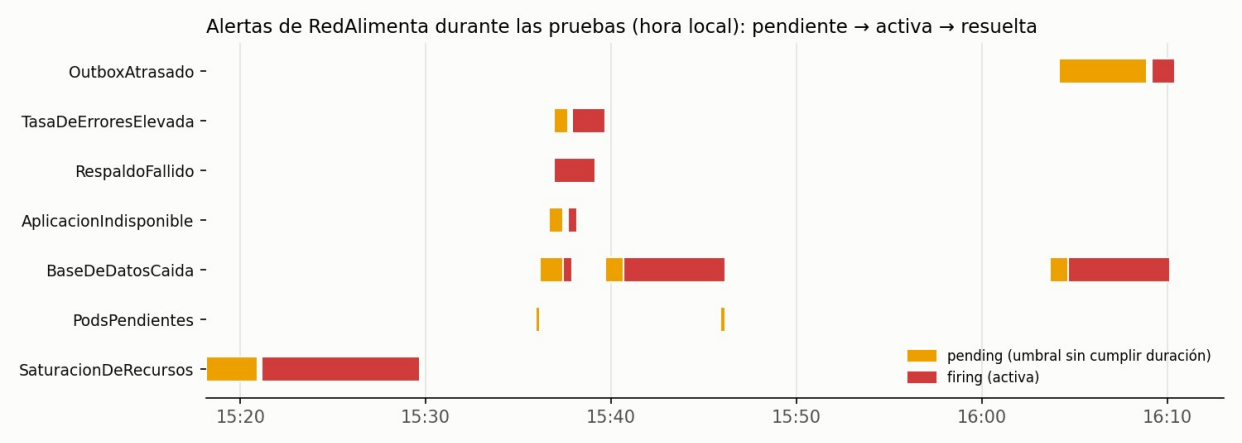
Logs centralizados en Loki (consulta {namespace="redalimenta", app="backend"})

Nota: en esta captura aún aparece failed to create fsnotify watcher: too many open files, consecuencia del límite de inotify del host. Se corrigió después con sysctl persistente: de 694 líneas en 25 min a 0 (evidencia SEC-03).

Alerta	Consulta (resumen)	Umbral / duración	Severidad	Acción
AplicacionIndisponible	kube_deployment_status_replicas_available{backend/frontend}	< 1 por 1 min	critical	describe deploy; eventos; readiness
NodoNoDisponible	kube_node_status_condition{Ready=true}	== 0 por 1 min	critical	nodo/kubelet; capacidad del otro worker
BaseDeDatosCaida	pg_up, mongodb_up, redalimenta_dependency_up	== 0 por 1 min	critical	StatefulSet, PVC, logs
TasaDeErroresElev	5xx / total	> 5 % por 1	warning	Loki por request_id; salud de

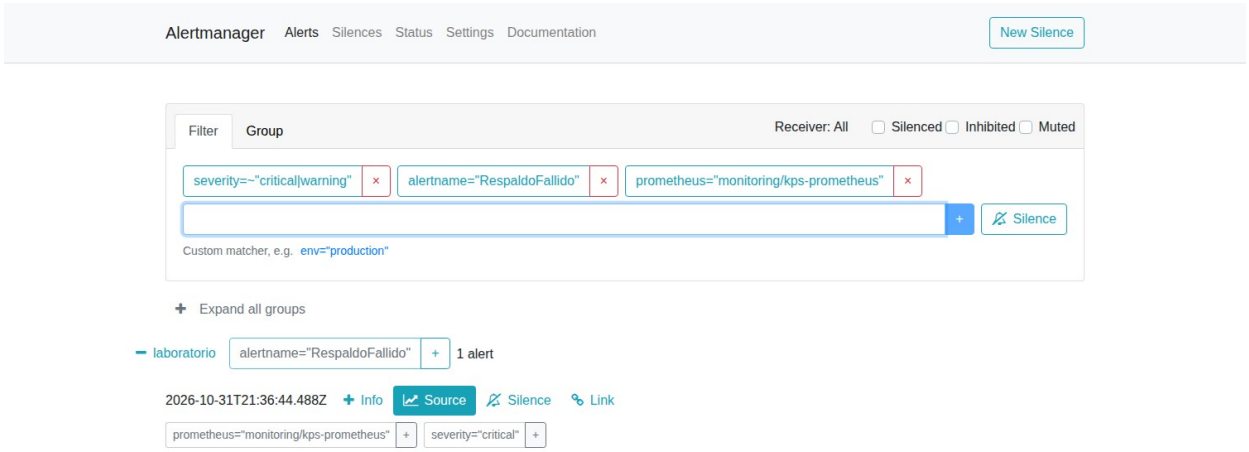
Alerta	Consulta (resumen)	Umbral / duración	Severidad	Acción
ada		min		BD
OutboxAtrasado	redalimenta_outbox_pendientes	> 50 por 5 min	warning	conectividad a MongoDB
SaturacionDeRecursos	HPA en máximo, CPU de nodo > 85 % o memoria > 90 %	3 min	warning	Pending, capacidad, BD
PodsPendientes	Pods Pending	> 0 por 5 min	warning	requests, cuota, topologySpread
RespaldoFallido	kube_job_status_failed{respaldo*}	> 0	critical	logs del job; repetir
RespaldoVencido	time() - last_successful_time	> 26 h	warning	respaldo manual
CertificadoPorExpirar	certmanager_certificate_expiration_timestamp_seconds	< 7 días	warning	Certificate/Issuer

Alertas provocadas de forma controlada (E10). Seis alertas pasaron por pendiente → activa → resuelta durante las pruebas, sin enviar datos a terceros (receptor local laboratorio). Historial extraído de la serie ALERTS de Prometheus:



Línea de tiempo de las alertas durante las pruebas

Alerta	Provocada por	Activa	Resuelta por
SaturacionDeRecursos	HPA en su máximo (6/6) durante la rampa	15:21:10–15:29:40	Fin de la carga y reducción del HPA
BaseDeDatosCaida	PostgreSQL a 0 réplicas; MongoDB a 0 (dos veces)	15:37:10–15:37:55; 15:40:40–15:46:10; 16:04:40–16:10:10	Restablecer el StatefulSet
AplicacionIndisponible	Readiness retira todas las réplicas con PostgreSQL caído	15:37:40–15:38:10	PostgreSQL de vuelta
TasaDeErroresElevada	503 durante la caída de PostgreSQL	15:37:55–15:39:40	Recuperación
RespaldoFallido	Job de respaldo lanzado con PostgreSQL caído	15:36:55–15:39:10	Diagnóstico (DNS sin endpoints), borrar el Job y repetir el respaldo (completo, sha256 OK)
OutboxAtrasado	80 eventos sin publicar más de 5 min con MongoDB caído (1.1.1)	16:09:10–16:10:25	Reconexión; outbox en 0



Alertmanager con la alerta RespaldoFallido activa (severidad critical, receptor local)

En la primera prueba con MongoDB caído (v1.0.0), **OutboxAtrasado no se activó**: el worker solo actualizaba la métrica `redalimenta_outbox_pendientes` cuando publicaba con éxito. Se corrigió en la 1.1.1 (se actualiza también en el camino de error) y la repetición sí activó la alerta.

Manual de incidentes

Escenario	Diagnóstico	Acciones	Verificación	Escalamiento
Pod de backend caído	`kubectl get pods`, `describe`, `logs --previous`	El Deployment lo recrea; si hay CrashLoop: rollback	readyReplicas = spec; sonda 200	Desarrollador si el fallo es de imagen
Mantenimiento de worker	`kubectl get pdb`; ubicación de las BD	`drain --ignore-daemonsets --delete-emptydir-data` y luego `uncordon`	Réplicas en el otro worker; outbox en 0	Planear ventana si el nodo aloja una BD
PostgreSQL caído	Alerta BaseDeDatosCaida; `readyz` 503; logs del StatefulSet	Revisar PVC/nodo; `rollout restart`; restaurar (D09) si hay corrupción	Endpoints del backend listos; reinicios sin cambio	DBA/alumno
MongoDB caído	OutboxAtrasado; rastreo 503	Restablecer el StatefulSet; el outbox se drena solo	Pendientes = 0; sin event_id duplicados	—
Errores 5xx elevados	Panel de errores; Loki `{app="backend"} | level="ERROR"`	Correlacionar request_id; rollback si coincide con un despliegue	Tasa < 1 %	Desarrollador
Respaldo fallido	`kubectl logs job/<respaldo> -c postgres/-c mongo`	Corregir la causa; `create job --from=cronjob/respaldo`	Job completo; sha256 OK	Operador

D09 · Respaldos, restauración y continuidad

Objetivos definidos antes de probar: RPO de 24 h (respaldo diario a las 02:00 America/Mexico_City) y RTO de 30 min (desde la decisión de restaurar hasta que la aplicación responde con datos restaurados).

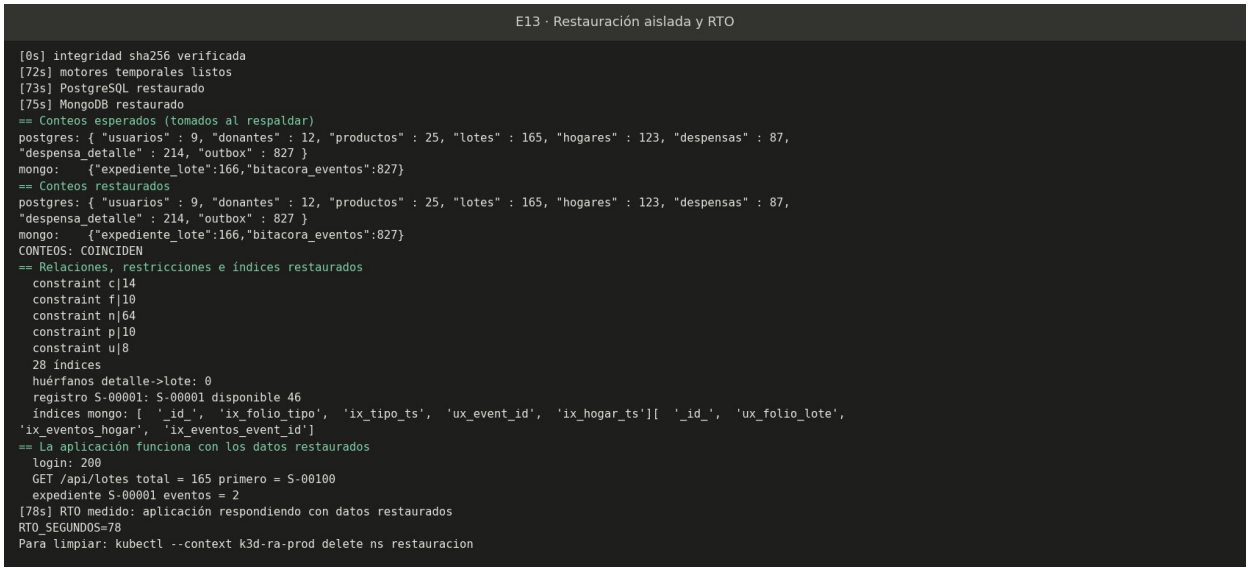
Método. El CronJob respaldo usa pg_dump -Fc, que es consistente porque trabaja sobre una instantánea transaccional. Para MongoDB usa mongodump --archive --gzip; como la instancia es standalone no hay --oplog, y la consistencia entre colecciones se completa reproyectando el outbox. Ambos se cifran con AES-256 y se guardan con sha256, conteos de referencia, duración y tamaño en un **PVC distinto** del de datos, con retención de 7 días y permisos 600 (umask 077). Además, respaldos/extraer.sh copia el último respaldo **fuera del clúster** (backups/, ignorado por Git). Una copia en caliente de archivos no se considera respaldo.

Ambiente	Fecha (UTC)	PostgreSQL	MongoDB	Integridad
prod	2026-10-31 20:22:07	103 408 B · 0 s	90 160 B · 0 s	sha256 OK
dev	2026-10-31 20:43:04	106 848 B · 1 s	94 208 B · 0 s	sha256 OK

Restauración aislada (E13). respaldos/restaurar.sh prod backups/prod/20261031T202207Z crea el namespace restauracion (PSA restricted, NetworkPolicy solo interna y sin ruta hacia las BD de origen), levanta PostgreSQL y MongoDB temporales, descifra **dentro** de los Pods, ejecuta pg_restore --no-owner --no-privileges y mongorestore --drop, y verifica:

- Conteos idénticos en los 8 conjuntos de PostgreSQL y en las 2 colecciones.
- 10 FK, 10 PK, 8 UNIQUE y 14 CHECK.
- 28 índices en PostgreSQL y los índices de Mongo.
- 0 huérfanos y el registro S-00001 presente.
- El backend (misma imagen) funcionando contra los datos restaurados: login 200, 165 lotes y expediente S-00001.

```
[0s] integridad sha256 verificada
[72s] motores temporales listos
[73s] PostgreSQL restaurado
[75s] MongoDB restaurado
CONTEOS: COINCIDEN
 login: 200 · GET /api/lotes total = 165 · expediente S-00001 eventos = 2
[78s] RTO medido: aplicación respondiendo con datos restaurados
```



Salida completa de la restauración aislada

Resultado: RTO medido de **78 s** (objetivo 30 min, cumple). La mayor parte del tiempo (72 s) fue la descarga y el arranque de los motores temporales; con las imágenes ya en caché bajaría a unos 20 s. **RPO:** el punto de recuperación es el último respaldo diario (peor caso 24 h, cumple). Para bajarlo, se propone archivado WAL continuo (pgBackRest) y respaldos cada hora del expediente.

Continuidad. Si se pierde el host, la recuperación sigue este orden: recrear los clústeres con los scripts (≈ 5 min), plataforma (≈ 8 min), secretos desde la copia de secrets/, desplegar, restaurar los respaldos extraídos y reproyectar el outbox.

D10 · Plan de pruebas, resultados y conclusiones

Matriz de aceptación

ID	Condición de aceptación	Resultado	Evidencia
E01	Dos contextos y topologías distintas; nodos y componentes sanos	**Cumple:** k3d-ra-dev (1 nodo) y k3d-ra-prod (1 + 2), todos Ready	E01-contextos-nodos.txt, captura term-E01
E02	URLs HTTPS distintas y datos independientes	**Cumple:** CA y URL por ambiente; un hogar creado en dev no existe en prod; secretos y PVC distintos	E02-independencia.txt
E03	CRUD de 3 entidades, roles y validaciones desde UI y API	**Cumple:** 22/22 en dev y en prod; sesión expirada → 401 (token vencido firmado dentro del clúster) y vigente → 200; capturas de la UI	E03-*.txt, capturas dev-*
E04	Uso funcional y persistente de ambas BD, trazable	**Cumple:** el mismo event_id está en outbox, bitácora y expediente; consultas con índice	E04-ambos-motores.txt
E05	Recreación de Pods de datos sin pérdida	**Cumple:** postgres-0 y mongo-0 recreados; mismos conteos y PVC	fallos/E05-*
E06	HPA crece por encima del mínimo y luego disminuye	**Cumple (2.° intento):** 2→6→2; el intento 1 falló por topologySpread y se corrigió	carga/
E07	Varias réplicas atienden; réplicas distribuidas	**Cumple:** 50/50 entre las réplicas de agent-0 y agent-1	E07-balanceo-prod.txt
E08	Conexiones permitidas y denegadas; DNS sano	**Cumple:** el intruso queda bloqueado y pasa a permitido con la etiqueta autorizada; firewall bloquea NodePort, kubelet y API desde un origen no autorizado	E08-*.txt
E09	Denegación por rol de app, RBAC y usuario de BD	**Cumple:** 403 de la app; luis.lector no puede leer Secrets; la app no puede hacer DDL; Mongo app sin createUser	E09-rbac-y-bd.txt
E10	Paneles con nodos, servicios y ambos motores; 2 alertas activadas y resueltas	**Cumple:** 6 alertas activadas y resueltas	fallos/E10-*
E11	Fallo de Pod y mantenimiento de worker con tiempos e impacto	**Cumple:** Pod 0 % de errores (Ready en 5 s); drain 0 % tras dejar Traefik en HA (intento 1: 34 s de corte)	fallos/E11-*
E12	Actualización y rollback verificados	**Cumple:** 1.0.0→1.1.1→1.0.0 con 0 errores tras el preStop (intento 1: 5 errores 502)	fallos/E12-*
E13	Respaldo y restauración verificada; RPO/RTO	**Cumple:** conteos idénticos, app funcionando; RTO 78 s; RPO ≤ 24 h	E13-*.txt
E14	Reconstrucción desde recursos limpios sin pasos tácitos ni secretos en Git	**Cumple:** dev reconstruido en 7 min 1 s y **prod en 12 min 3 s** (3 nodos, plataforma, observabilidad, datos, app 1.1.1 y firewall), 22/22 en ambos; 0 archivos sensibles versionables	E14-reconstruccion-dev.txt, E14-reconstruccion-prod.txt

Bitácora de pruebas fallidas y repeticiones (no se sustituyeron mediciones):

- E06 intento 1: 4 réplicas en Pending.
- E11 drain intento 1: 34.2 s sin servicio por Traefik con una sola réplica.
- E12 intento 1: 5 errores (502 y conexión) por no tener preStop.
- E11 MongoDB intento 1: OutboxAtrasado sin activarse y reproyectar ausente en la 1.0.0.
- Primer build 1.0.0 rechazado por Trivy.

Todos los intentos se conservan en docs/evidencias/ con el sufijo intento1 o RECHAZADO.

E02 · Ambientes independientes

```
# 2026-10-31T20:48:17Z
== dev: https://redalimenta-dev.172-30-0-200.sslip.io -> {"version":"1.1.0","ambiente":"dev"}
   subject=
   issuer=CN=RedAlimenta Lab CA (dev)
   Verify return code: 0 (ok)
       Verify return code: 0 (ok)
== prod: https://redalimenta.172-31-0-200.sslip.io -> {"version":"1.0.0","ambiente":"prod"}
   subject=
   issuer=CN=RedAlimenta Lab CA (prod)
   Verify return code: 0 (ok)
       Verify return code: 0 (ok)
== El certificado de prod NO es válido con la CA de dev: 000 (000 = verificación TLS fallida)
== Datos independientes: se crea un hogar solo en dev
   creado IND-144817 en dev
   dev: hogares con clave IND-144817 = 1 · total hogares = 127
   prod: hogares con clave IND-144817 = 0 · total hogares = 123
== Credenciales distintas (huella SHA-256 truncada del secreto de sesión, sin revelarlo)
   dev: 6f0c15547b0a
   prod: 4bfa0b7ca32b
== Volúmenes distintos
   dev:
     datos-mongo-0      pvc-6035c089-31cc-4489-a30f-acff246944ab   2Gi
     datos-postgres-0   pvc-e5f93308-f4b2-485b-a8e6-f25131b80665   2Gi
     respaldos          pvc-8a5eae86-77ef-4344-820b-8c4b12eae292   5Gi
   prod:
     datos-mongo-0      pvc-002089b0-3d18-4925-9ebb-762581e4e41f   5Gi
     datos-postgres-0   pvc-494a3137-d380-4b21-a3ce-e3960de37371   5Gi
     respaldos          pvc-7946f9f8-352a-428f-bac7-01d6cbc6022a   5Gi
```

Ambientes independientes: URL, CA, datos, credenciales y volúmenes

E07 · Balanceo entre réplicas (prod)

```
# E07 Balanceo 2026-10-31T20:25:55Z — DIAG_REPLICA activado solo durante la prueba
== Réplicas backend y nodo
backend-5684f657db-lp6gh   k3d-ra-prod-agent-1   10.52.147.33
backend-5684f657db-nfqst   k3d-ra-prod-agent-0   10.52.3.23
== 100 solicitudes HTTPS, conexión nueva cada una (MetalLB -> Traefik -> réplicas)
     50 backend-5684f657db-lp6gh
     50 backend-5684f657db-nfqst
== Identificador retirado: HTTP 404 (404 esperado)
== Distribución entre workers (topologySpreadConstraints maxSkew=1)
backend-674bd556db-8l944    k3d-ra-prod-agent-0
frontend-69969587b9-k6lgd   k3d-ra-prod-agent-0
backend-674bd556db-ltnh5    k3d-ra-prod-agent-1
frontend-69969587b9-56pgl   k3d-ra-prod-agent-1
```

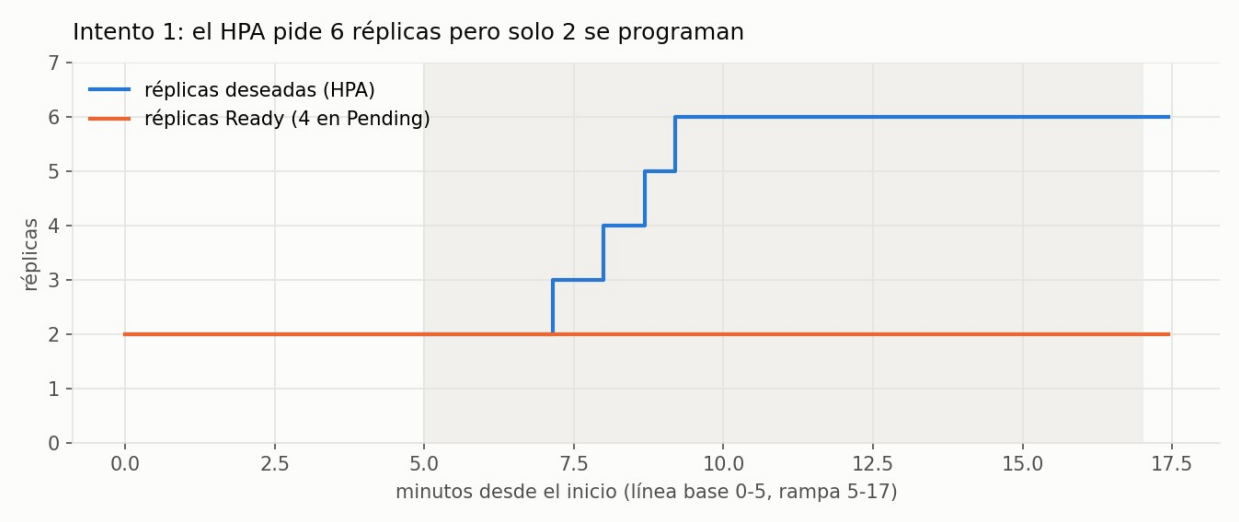
Balanceo: 100 solicitudes repartidas entre réplicas en workers distintos

Experimento de escalado horizontal (E06)

Diseño (fijado antes de medir).

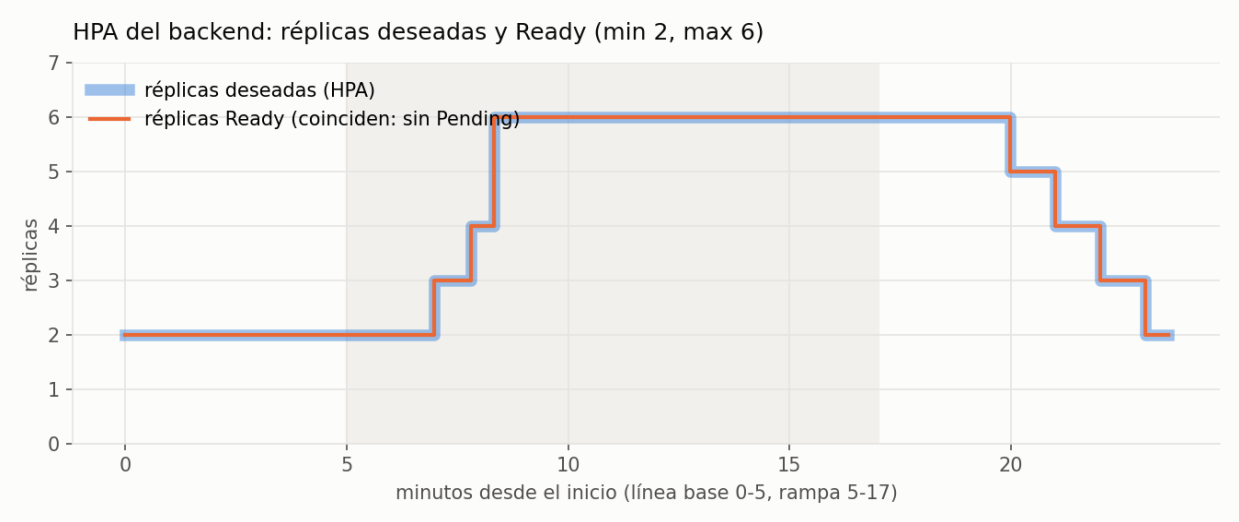
- **Transacción representativa:** búsqueda paginada de lotes (ILIKE + count + JOIN) y listado de hogares, con sesión de coordinador; no mide archivos estáticos.
- **Generador:** k6 v1.8.2 (grafana/k6:1.8.2) en el equipo administrador, fuera del clúster, por HTTPS a través de MetalLB → Traefik. Script: tests/carga/escenario.js.
- **Etapas:** línea base de 5 min con 2 VU, luego rampa de 12 min (10 → 30 → 60 VU, 3 min sostenidos en 60 y bajada).
- **Umbral acordado:** p95 < 1 s y errores < 1 %.
- **Serie temporal:** muestreo.py registra cada 10 s réplicas deseadas, actuales y Ready, Pending, CPU (HPA y Prometheus), RPS, p95, errores y conexiones a PostgreSQL. La API de métricas se verificó antes (kubectl top, condición ScalingActive).

Intento 1: falló el escalado. El HPA calculó 6 réplicas, pero **4 quedaron en Pending**. Diagnóstico con kubectl describe pod: *"2 node(s) didn't match pod topology spread constraints, 1 node(s) had untolerated taint(s)"*. La causa: con nodeTaintsPolicy por defecto (Ignore), el control plane con taint cuenta como dominio con 0 Pods, así que maxSkew: 1 limitaba a una réplica por worker. **Cambio justificado:** nodeTaintsPolicy: Honor en patch-replicas-distribucion.yaml. El intento se conserva en serie-con-hpa-intento1-pending.csv.

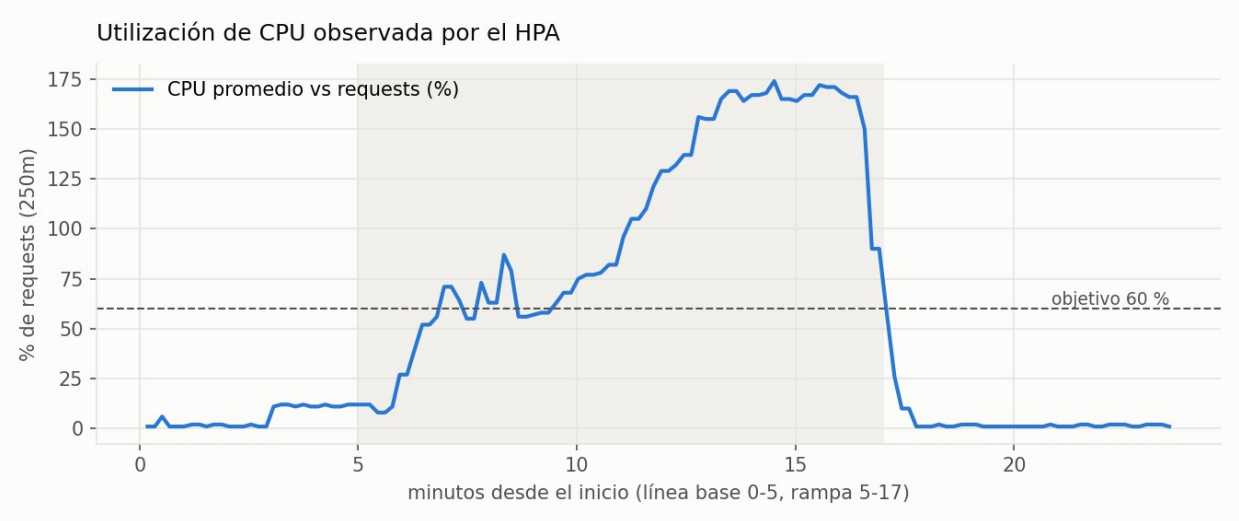


Intento 1: réplicas deseadas frente a Ready (4 Pending)

Repetición (con HPA corregido). El HPA escaló 2 → 3 → 4 → 6 entre los minutos 7 y 8 de la corrida, con el CPU promedio por encima del 60 %. Las **6 réplicas quedaron Ready** (3 por worker, 0 Pending). Tras la carga, esperó la ventana de estabilización (180 s) y bajó 1 Pod por minuto hasta 2. Los eventos SuccessfulRescale están en eventos-hpa-con-hpa-v2.txt.



Réplicas deseadas y Ready durante la corrida con HPA



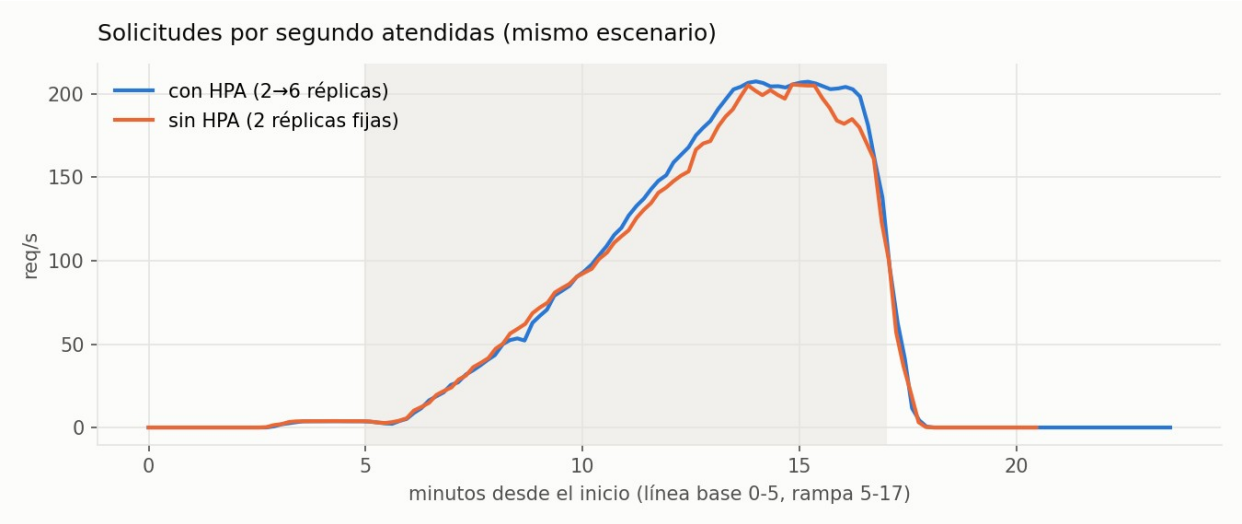
Utilización de CPU vista por el HPA

Métrica (rampa de 12 min, mismo escenario)	Con HPA (2→6)	Sin HPA (2 fijas)	Intento 1 (2 Ready, 4 Pending)
Solicitudes totales / errores	86 299 / **0 %**	82 909 / **0 %**	85 247 / 0 %
Throughput promedio	119.7 req/s	115.1 req/s	118.3 req/s
Latencia p50 / p95 / p99	25.5 / **99.7** / 150.1 ms	26.8 / **150.0** / 200.9 ms	21.8 / 125.4 / 177.0 ms
Réplicas Ready máximas	6	2	2
CPU backend máximo (suma)	2.52 cores	2.06 cores	2.07 cores
CPU PostgreSQL máximo	1.03 cores	0.81 cores	0.79 cores
Conexiones PostgreSQL máximas	39	22	22
Memoria backend máxima (suma)	523 MiB	200 MiB	194 MiB
Línea base (5 min, 2 VU) p95	36.6 ms	32.8 ms	—

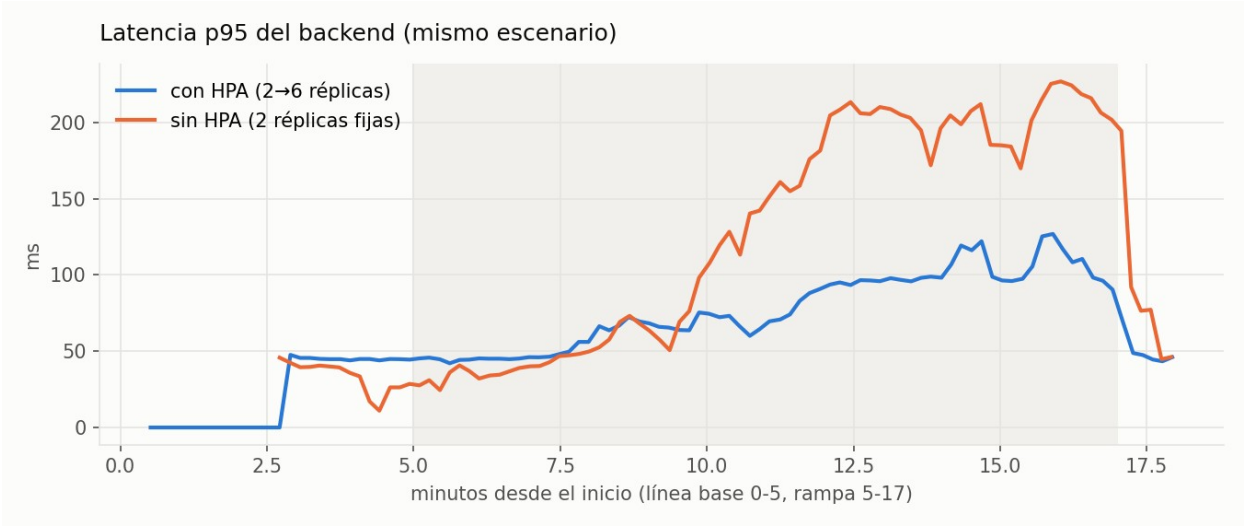
E06 · kubectl describe hpa backend

```
Name:                                                  backend
Namespace:                                             redalimenta
Labels:                                                <none>
Annotations:                                           <none>
CreationTimestamp:                                     Sat, 31 Oct 2026 15:09:03 -0600
Reference:                                             Deployment/backend
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  1% (4m) / 60%
Min replicas:                                          2
Max replicas:                                          6
Behavior:
  Scale Up:
    Stabilization Window: 0 seconds
    Select Policy: Max
    Policies:
      - Type: Pods  Value: 2  Period: 30 seconds
  Scale Down:
    Stabilization Window: 180 seconds
    Select Policy: Max
    Policies:
      - Type: Pods  Value: 1  Period: 60 seconds
Deployment pods:    2 current / 2 desired
Conditions:
  Type            Status  Reason            Message
  ----            ------  ------            -------
  AbleToScale     True    ReadyForNewScale  recommended size matches current size
  ScalingActive   True    ValidMetricFound  the HPA was able to successfully calculate a replica count from cpu resource
utilization (percentage of request)
  ScalingLimited  True    TooFewReplicas    the desired replica count is less than the minimum replica count
  ScaledToZero    False   NotScaledToZero   the HPA controller did not scale the workload to zero
Events:
  Type     Reason                        Age    From                       Message
  ----     ------                        ----   ----                       -------
  Warning  FailedGetResourceMetric       23m    horizontal-pod-autoscaler  failed to get cpu utilization: did not receive
metrics for targeted pods (pods might be unready)
  Warning  FailedComputeMetricsReplicas  23m    horizontal-pod-autoscaler  invalid metrics (1 invalid out of 1), first error is:
 failed to get cpu resource metric value: failed to get cpu utilization: did not receive metrics for targeted pods (pods might
be unready)
  Normal   SuccessfulRescale             16m    horizontal-pod-autoscaler  New size: 3; reason: cpu resource utilization
… (9 líneas más en describe-hpa-con-hpa-v2.txt)
```

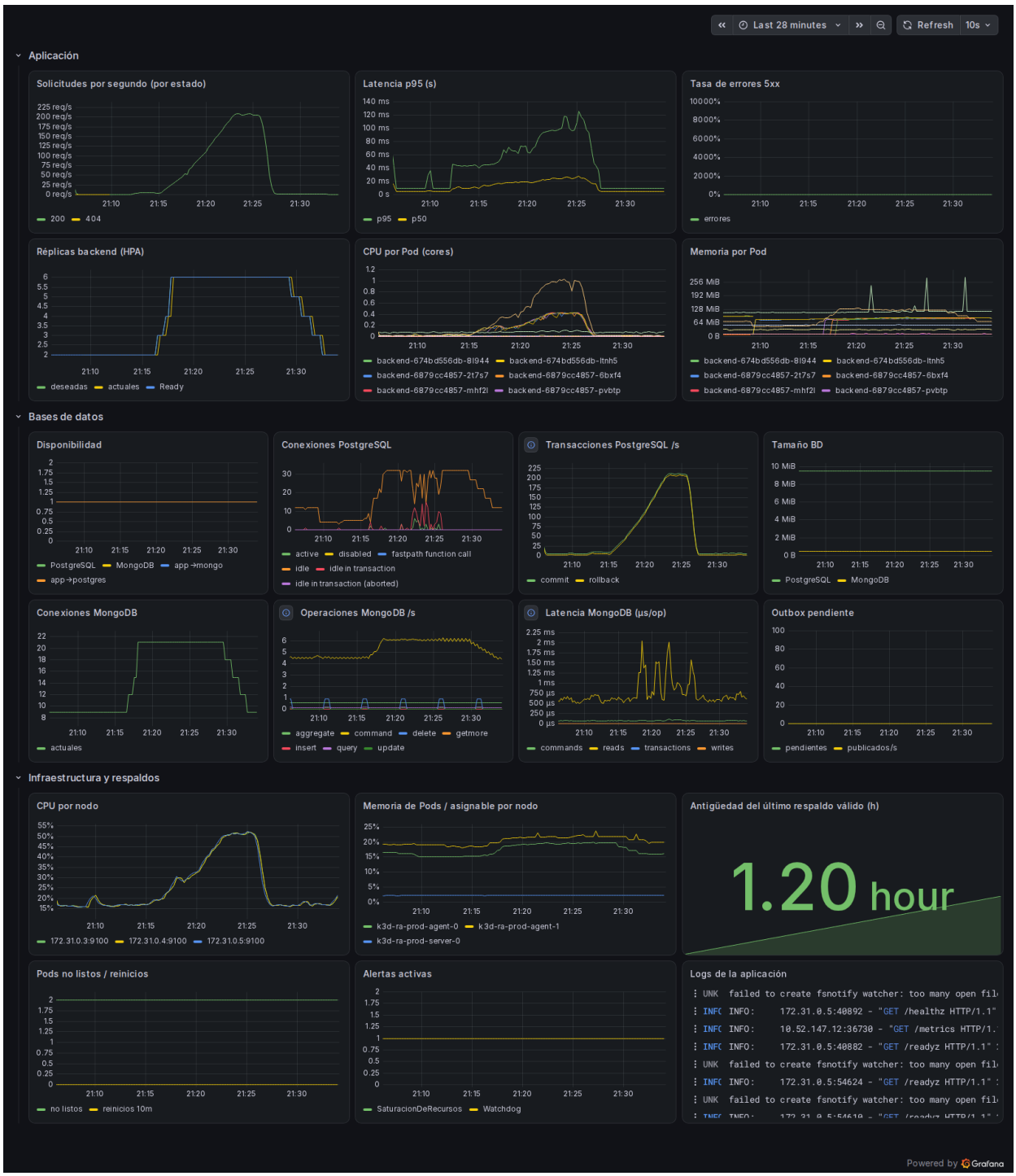
kubectl describe hpa tras la repetición: condiciones y eventos de escalado



RPS con y sin HPA



p95 con y sin HPA



Panel de Grafana durante la corrida con HPA (réplicas, CPU por Pod, conexiones y transacciones)

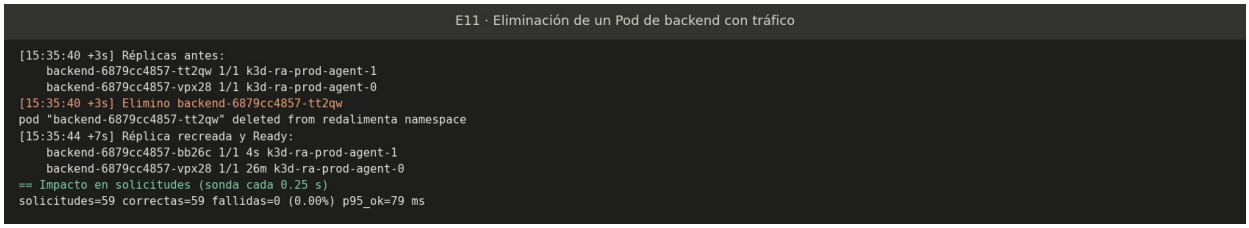
Interpretación.

- Ambas configuraciones cumplen el umbral (p95 < 1 s, 0 % de errores). Con HPA, el p95 bajó un **34 %** (150 → 99.7 ms) y el p99 un 25 %.
- El throughput casi no cambió (+4 %). El generador es de **lazo cerrado** (cada VU espera la respuesta y 0.5 s), así que con 60 VU la demanda está acotada en ~120 req/s. Las réplicas extra reducen la cola y la latencia, pero no generan más demanda. **La mejora no es lineal** en número de Pods.
- El siguiente cuello de botella es PostgreSQL: pasó de 0.8 a 1.03 cores, y las conexiones de 22 a 39 (dentro del presupuesto de 60 + outbox y del `max_connections=100`).
- Con 6 réplicas, los requests del backend suman 1.5 vCPU y caben en los 8 vCPU asignables de los workers. Si se superara la capacidad, las réplicas quedarían en Pending (alerta `PodsPendientes`), porque el HPA no crea nodos.

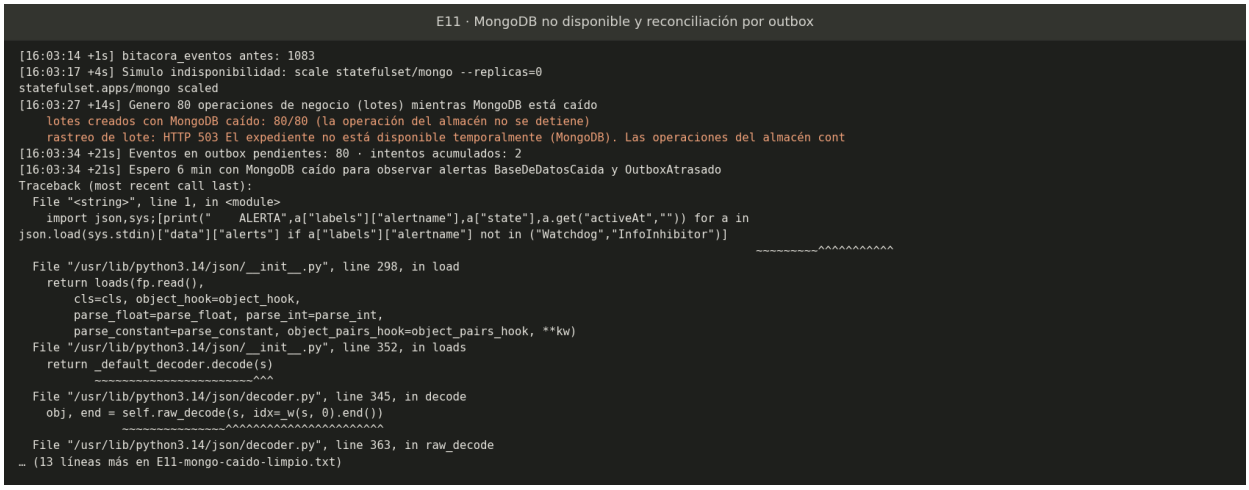
Pruebas de fallos (E11), actualización (E12) y persistencia (E05)

Cada prueba se ejecutó con una sonda de tráfico continuo (tests/fallos/sonda.py: GET /api/lotes autenticado cada 0.25 s por HTTPS) y una línea de tiempo (tests/fallos/fallos.sh).

Prueba	Acción	Recuperación	Impacto medido
Pod de backend	kubectl delete pod de 1 de 2 réplicas	Réplica nueva Ready en 5 s, en el mismo worker (distribución)	**0/59 errores**, p95 79 ms
Pods de datos (E05)	Borrar postgres-0 y mongo-0	Recreados en el mismo nodo con el mismo PVC	165 lotes, 87 despensas y 166 expedientes antes y después
PostgreSQL no disponible	scale statefulset/postgres --replicas=0 durante 90 s	Backend Ready 10 s después de restablecer la BD; **0 reinicios** de contenedores (la liveness no depende de la BD)	369/417 solicitudes 503 durante 99 s (esperado: es la fuente de verdad). Traefik responde "no available server" porque la readiness retira las réplicas
MongoDB no disponible (1.1.1)	scale statefulset/mongo --replicas=0 durante 6 min y 80 altas de lotes	Outbox drenado en 31 s tras la reconexión; bitácora 1083 → 1163 = 1163 eventos publicados; **0 event_id duplicados**; reproyectar 172 eventos tampoco duplicó	**0/1210 errores**; 80/80 operaciones de negocio OK; el rastreo responde 503 con mensaje claro
Mantenimiento de worker	drain agent-1 --ignore-daemonsets --delete-emptydir-data (sin --force), 45 s y uncordon	Réplicas y Traefik pasan a agent-0 respetando los PDB; mongo-0 en Pending (PV local) hasta el uncordon, de vuelta en 7 s; rebalanceo con rollout restart	**0/280 errores** (intento 1 con Traefik de 1 réplica: 35 errores, 34 s)
Actualización y rollback	Promover 1.1.1 (mismo digest de dev) y rollout undo	1.1.1 en 21 s; undo a 1.0.0 en 14 s; migración 0002 se mantiene (compatible)	**0/128 errores**; 22/22 pruebas tras el rollback (intento 1 sin preStop: 5 errores en 35 s)



Eliminación de un Pod de backend con tráfico



MongoDB no disponible: la operación sigue y la reconciliación no duplica

E11 · Mantenimiento de worker (drain/uncordon)

```
[16:01:41 +3s] Antes del mantenimiento (k3d-ra-prod-agent-1 aloja mongo-0 con volumen local):
    backend-66ddf79494-948lp Running k3d-ra-prod-agent-0
    backend-66ddf79494-tvsgh Running k3d-ra-prod-agent-1
    frontend-54579c8f5-fgp2g Running k3d-ra-prod-agent-0
    frontend-54579c8f5-nmfhh Running k3d-ra-prod-agent-1
    migraciones-qdpq4 Completed k3d-ra-prod-agent-0
    mongo-0 Running k3d-ra-prod-agent-1
    mongodb-exporter-68ff85dbff-wph8z Running k3d-ra-prod-agent-0
    outbox-77b5db9f88-4zf92 Running k3d-ra-prod-agent-0
    postgres-0 Running k3d-ra-prod-agent-0
    postgres-exporter-76d8c7b59d-2jnjz Running k3d-ra-prod-agent-0
NAME       MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
backend    1               N/A               1                     109m
frontend   1               N/A               1                     109m
[16:01:42 +4s] kubectl drain k3d-ra-prod-agent-1 --ignore-daemonsets --delete-emptydir-data (sin --force; respeta PDB)
    node/k3d-ra-prod-agent-1 cordoned
    node/k3d-ra-prod-agent-1 drained
[16:01:50 +12s] Durante el mantenimiento:
    backend-66ddf79494-948lp Running k3d-ra-prod-agent-0
    backend-66ddf79494-gpwr2 Running k3d-ra-prod-agent-0
    frontend-54579c8f5-fgp2g Running k3d-ra-prod-agent-0
    frontend-54579c8f5-w4xj6 Running k3d-ra-prod-agent-0
    migraciones-qdpq4 Completed k3d-ra-prod-agent-0
    mongo-0 Pending <none>
    mongodb-exporter-68ff85dbff-wph8z Running k3d-ra-prod-agent-0
    outbox-77b5db9f88-4zf92 Running k3d-ra-prod-agent-0
    postgres-0 Running k3d-ra-prod-agent-0
    postgres-exporter-76d8c7b59d-2jnjz Running k3d-ra-prod-agent-0
    k3d-ra-prod-agent-0    Ready                  <none>          150m   v1.37.1+k3s1
    k3d-ra-prod-agent-1    Ready,SchedulingDisabled   <none>          150m   v1.37.1+k3s1
    k3d-ra-prod-server-0   Ready                  control-plane   150m   v1.37.1+k3s1
[16:01:50 +12s] mongo-0 queda Pending: su PV local-path tiene afinidad a k3d-ra-prod-agent-1 (limitación documentada). La API
sigue atendiendo:
    GET /api/version durante el drain: HTTP 200
[16:02:35 +57s] Outbox pendiente durante la ventana: 0
[16:02:35 +57s] kubectl uncordon k3d-ra-prod-agent-1
node/k3d-ra-prod-agent-1 uncordoned
[16:02:42 +64s] mongo-0 de vuelta en k3d-ra-prod-agent-1
[16:02:43 +65s] Outbox pendiente tras la recuperación: 0
[16:02:43 +65s] Rebalanceo tras el mantenimiento (el scheduler no mueve Pods ya en ejecución): rollout restart
[16:02:58 +80s] Distribución final:
    backend-7687b4dfc5-d27lb Running k3d-ra-prod-agent-1
    backend-7687b4dfc5-fk4qh Running k3d-ra-prod-agent-0
    frontend-6759ffcdd8-64f8b Running k3d-ra-prod-agent-1
    frontend-6759ffcdd8-jrzsk Running k3d-ra-prod-agent-0
    migraciones-qdpq4 Completed k3d-ra-prod-agent-0
    mongo-0 Running k3d-ra-prod-agent-1
    mongodb-exporter-68ff85dbff-wph8z Running k3d-ra-prod-agent-0
    outbox-77b5db9f88-4zf92 Running k3d-ra-prod-agent-0
    postgres-0 Running k3d-ra-prod-agent-0
    postgres-exporter-76d8c7b59d-2jnjz Running k3d-ra-prod-agent-0
… (2 líneas más en E11-drain-worker.txt)
```

Mantenimiento de worker con drain y uncordon

E12 · Actualización 1.0.0 → 1.1.1 y rollback

```
[15:56:14 +4s] Versión en producción: {"version":"1.0.0","ambiente":"prod"}
10        <none>
11        <none>

[15:56:15 +4s] Promuevo el artefacto aprobado en dev (1.1.1) — mismo digest, solo cambia la configuración del overlay
    digest: sha256:3a862c988a485498e8103f325c8690e4bd850cb459f1b6bd244fdde97557c2f2
    digest: sha256:bf54d51d36e8b20bcc05aa4b90c3b84aaf6b4cb973284077db9a731ee3cae15e
[15:56:32 +21s] Versión tras rolling update: {"version":"1.1.1","ambiente":"prod"} · migración: 0002
    backend-58d879bd4b-jxdn4   k3d-ra-registry.localhost:5050/redalimenta-
backend@sha256:3a862c988a485498e8103f325c8690e4bd850cb459f1b6bd244fdde97557c2f2
    backend-58d879bd4b-p9w64   k3d-ra-registry.localhost:5050/redalimenta-
backend@sha256:3a862c988a485498e8103f325c8690e4bd850cb459f1b6bd244fdde97557c2f2
[15:56:32 +21s] Rollback: kubectl rollout undo deploy/backend y deploy/frontend (el esquema 0002 es compatible con 1.0.0; no se
revierte)
deployment.apps/backend rolled back
deployment.apps/frontend rolled back
[15:56:46 +35s] Versión tras rollback: {"version":"1.0.0","ambiente":"prod"} · migración sigue en 0002
    8         <none>
    12        <none>
    13        <none>

    pruebas funcionales tras rollback: 22 passed in 1.43s
== Impacto en solicitudes (sonda cada 0.25 s)
solicitudes=128 correctas=128 fallidas=0 (0.00%) p95_ok=86 ms
```

Actualización y rollback con versión y funcionalidad verificadas

E14 · Reconstrucción de desarrollo desde cero

```
# E14 Reconstrucción de desarrollo desde recursos limpios 2026-10-31T22:03:34Z
Precondición: se elimina el clúster ra-dev (nodos, volúmenes, Secrets, CA). Se conservan: repositorio, registro OCI y
secrets/dev.env (copia de recuperación).
INFO[0001] Successfully deleted cluster ra-dev!
[2s] clúster eliminado
NAME                 STATUS   ROLES           AGE   VERSION        INTERNAL-IP   EXTERNAL-IP   OS-IMAGE           KERNEL-
VERSION              CONTAINER-RUNTIME
k3d-ra-dev-server-0   Ready    control-plane   90s   v1.37.1+k3s1   172.30.0.3    <none>        K3s v1.37.1+k3s1
7.0.0-38-generic (amd64)   containerd://2.3.4-k3s1
[100s] paso 2 listo (clúster + Calico)
NAME      TYPE           CLUSTER-IP    EXTERNAL-IP    PORT(S)                      AGE
traefik   LoadBalancer   10.43.88.97   172.30.0.200   80:31388/TCP,443:32595/TCP   27s
[229s] paso 3 listo (MetalLB, cert-manager, Traefik)
configmap/ca-laboratorio aplicado
[231s] paso 4 listo (Secrets)
Semilla aplicada: {'usuarios': 9, 'donantes': 12, 'productos': 20, 'lotes': 160, 'hogares': 120, 'despensas': 85,
'despensa_detalle': 204, 'outbox': 788}
deployment "backend" successfully rolled out
deployment "outbox" successfully rolled out
deployment "frontend" successfully rolled out
[418s] pasos 5-6 listos (datos, migraciones, app)
k3d-ra-dev-server-0: firewall aplicado
[418s] firewall aplicado
versión: {"version":"1.1.1","ambiente":"dev"}
pruebas funcionales: 22 passed in 1.33s
TIEMPO_TOTAL=421s
== Secretos en Git: 0 archivos (esperado 0)
== Verificación .gitignore (git add --dry-run): archivos sensibles que se agregarían
0
… (6 líneas más en E14-reconstruccion-dev.txt)
```

Reconstrucción de desarrollo desde cero con el manual (E14)

E14 · Reconstrucción de PRODUCCIÓN desde cero

```
# E14 Reconstrucción de PRODUCCIÓN desde recursos limpios 2026-10-31T22:33:18Z
Precondición: se elimina ra-prod (nodos, PVC, Secrets, CA, observabilidad). Se conservan: repositorio, registro OCI,
secrets/prod.env y respaldos extraídos.
INFO[0005] Successfully deleted cluster ra-prod!
[6s] clúster eliminado
NAME                    STATUS   ROLES           AGE     VERSION        INTERNAL-IP   EXTERNAL-IP   OS-IMAGE           KERNEL-
VERSION              CONTAINER-RUNTIME
k3d-ra-prod-agent-0    Ready    <none>          2m31s   v1.37.1+k3s1   172.31.0.4    <none>        K3s v1.37.1+k3s1
7.0.0-38-generic (amd64)   containerd://2.3.4-k3s1
k3d-ra-prod-agent-1    Ready    <none>          2m31s   v1.37.1+k3s1   172.31.0.5    <none>        K3s v1.37.1+k3s1
7.0.0-38-generic (amd64)   containerd://2.3.4-k3s1
k3d-ra-prod-server-0   Ready    control-plane   2m34s   v1.37.1+k3s1   172.31.0.3    <none>        K3s v1.37.1+k3s1
7.0.0-38-generic (amd64)   containerd://2.3.4-k3s1
[169s] paso 2: clúster 1+2 y Calico
NAME      TYPE           CLUSTER-IP    EXTERNAL-IP    PORT(S)                      AGE
traefik   LoadBalancer   10.53.43.49   172.31.0.200   80:31500/TCP,443:31642/TCP   28s
[302s] paso 3: MetalLB, cert-manager, Traefik (2 réplicas + PDB)
prometheus-kps-prometheus-0               2/2     Running   0          4m12s
[585s] observabilidad
configmap/ca-laboratorio aplicado
[587s] paso 4: Secrets
Semilla aplicada: {'usuarios': 9, 'donantes': 12, 'productos': 20, 'lotes': 160, 'hogares': 120, 'despensas': 85,
'despensa_detalle': 204, 'outbox': 788}
deployment "backend" successfully rolled out
deployment "outbox" successfully rolled out
deployment "frontend" successfully rolled out
[720s] pasos 5-7: datos, migraciones, app (overlay con digest 1.1.1)
k3d-ra-prod-agent-0: firewall aplicado
[720s] firewall
versión: {"version":"1.1.1","ambiente":"prod"}
pruebas funcionales: 22 passed, 1 skipped in 1.48s
   backend-58d879bd4b-9c2cv Running k3d-ra-prod-agent-1
   backend-58d879bd4b-v84cn Running k3d-ra-prod-agent-0
   frontend-6f4d88d4c5-pd4s5 Running k3d-ra-prod-agent-0
   frontend-6f4d88d4c5-tkskg Running k3d-ra-prod-agent-1
   migraciones-jkk2d Completed 2m15s
   mongo-0 Running k3d-ra-prod-agent-1
   mongodb-exporter-68ff85dbff-k5rjq Running k3d-ra-prod-agent-0
   outbox-77b5db9f88-chjcq Running k3d-ra-prod-agent-1
… (6 líneas más en E14-reconstruccion-prod.txt)
```

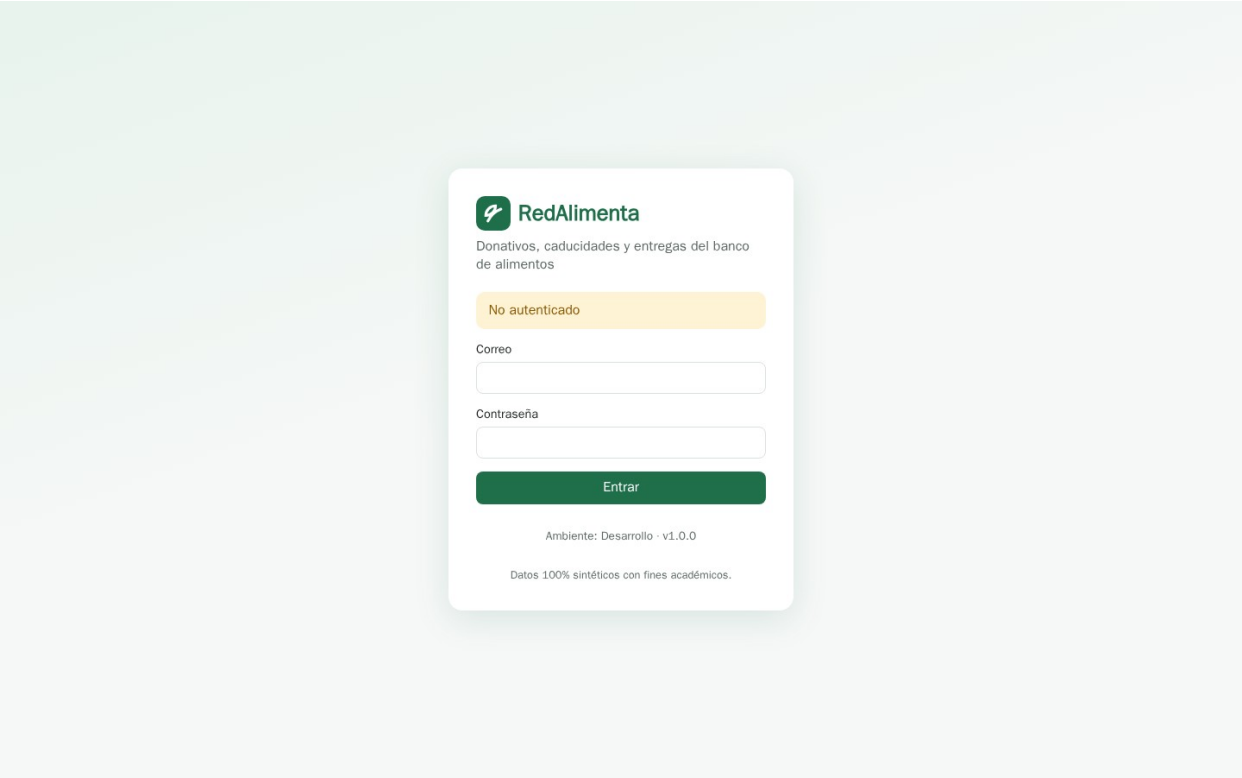
Reconstrucción de producción desde cero: 12 min 3 s y 22/22 pruebas (E14)

Qué sigue funcionando ante cada fallo (pregunta de defensa).

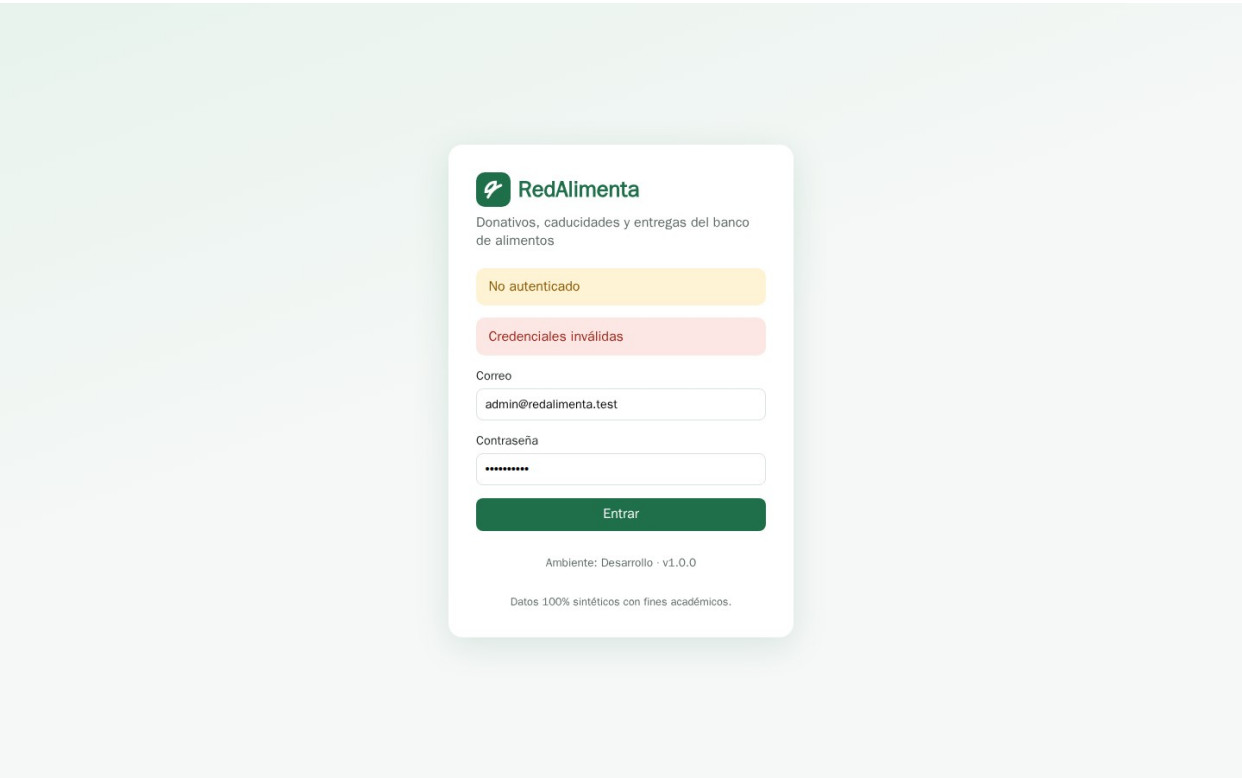
- **Pod:** todo; el Service deja de enviarle tráfico y el Deployment lo reemplaza.
- **Worker:** la aplicación (réplicas y Traefik en el otro worker). La base cuyo PV local estaba en ese worker queda fuera hasta que vuelve: si es MongoDB, la app sigue y el outbox acumula; si es PostgreSQL, la app no atiende.
- **Control plane:** las cargas en ejecución y el tráfico siguen (kube-proxy y Traefik ya tienen sus reglas), pero no hay reprogramación, HPA, despliegues ni drain.
- **Almacenamiento:** si se pierde el volumen de una base, se restaura desde el respaldo (RTO medido 78 s; RPO hasta 24 h).

Mejora identificada. Con PostgreSQL caído, la readiness retira todas las réplicas y el usuario recibe el 503 genérico de Traefik en lugar del mensaje JSON de la app. Alternativa evaluada: una readiness que no dependa de PostgreSQL, con el error manejado en la app. La desventaja es enviar tráfico a réplicas que no pueden servirlo. Se mantiene el diseño actual y se documenta.

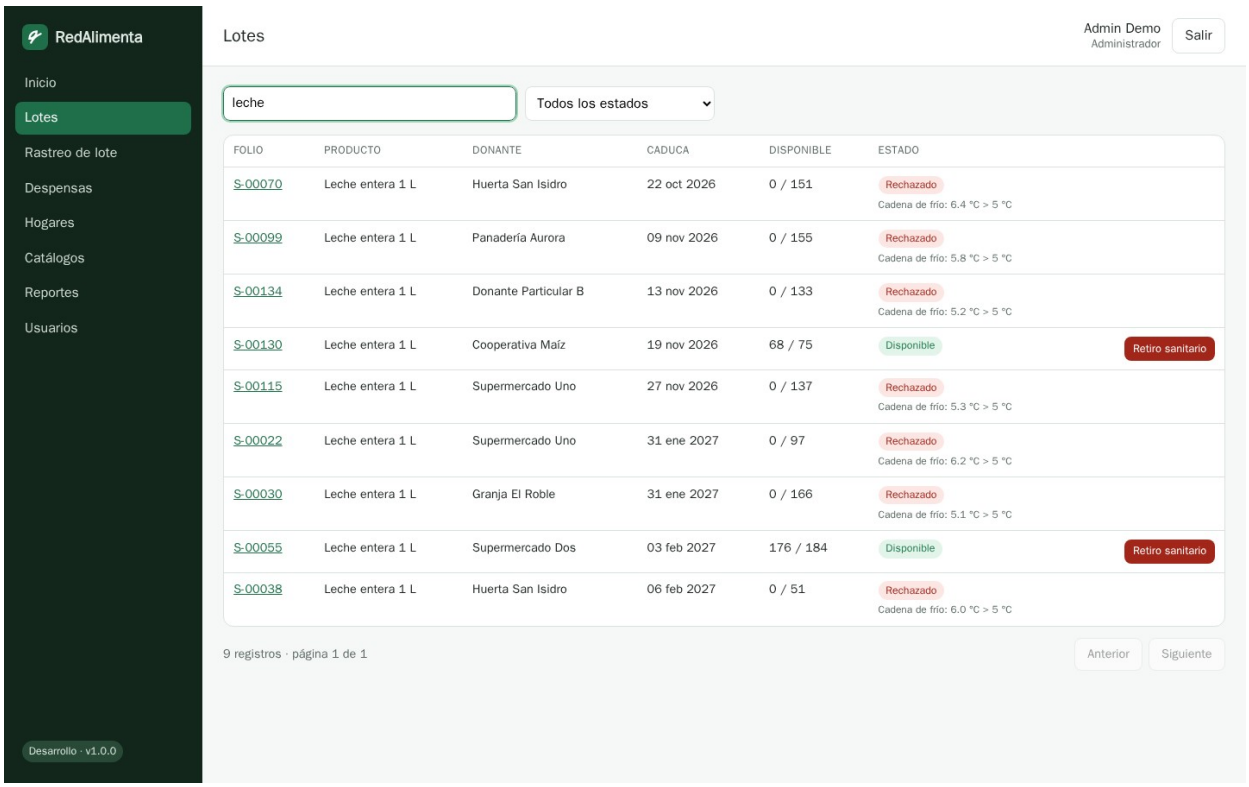
Capturas de la aplicación



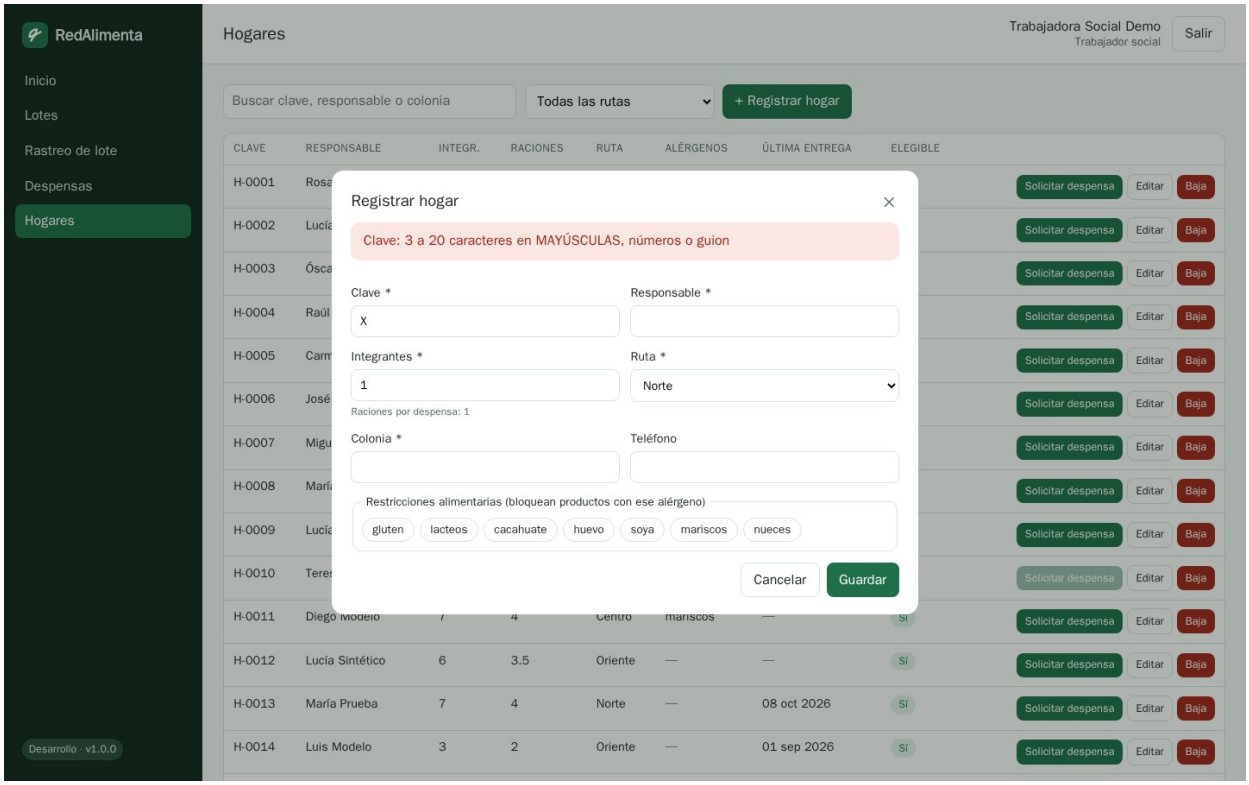
Inicio de sesión con indicación del ambiente



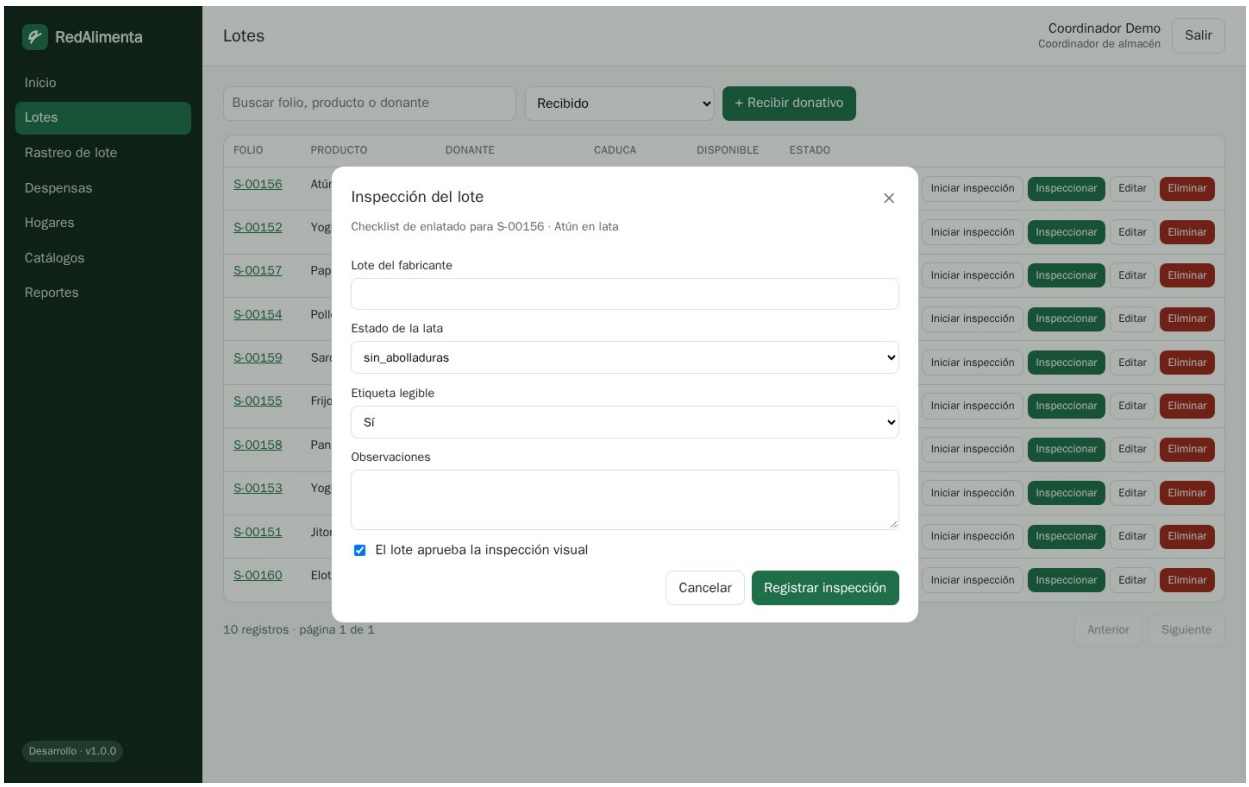
Mensaje de error de autenticación genérico



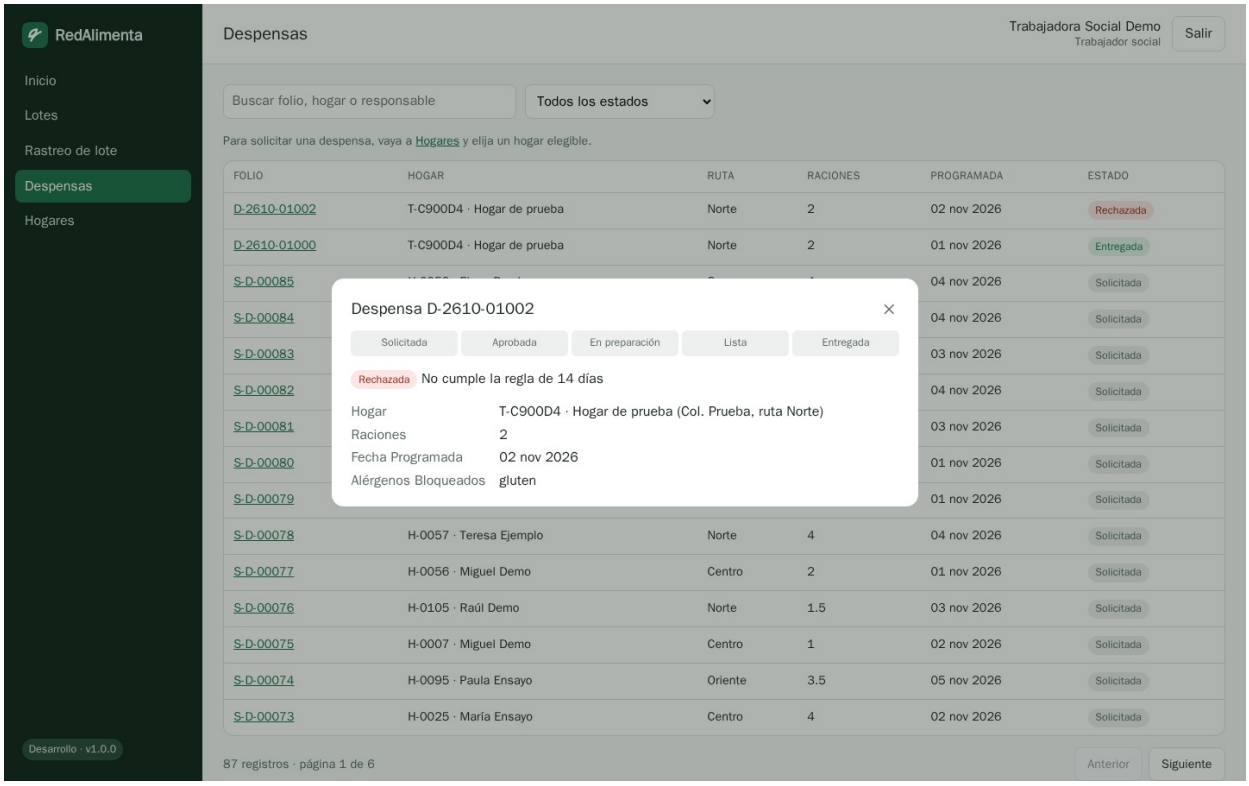
Lotes con búsqueda, estados y paginación



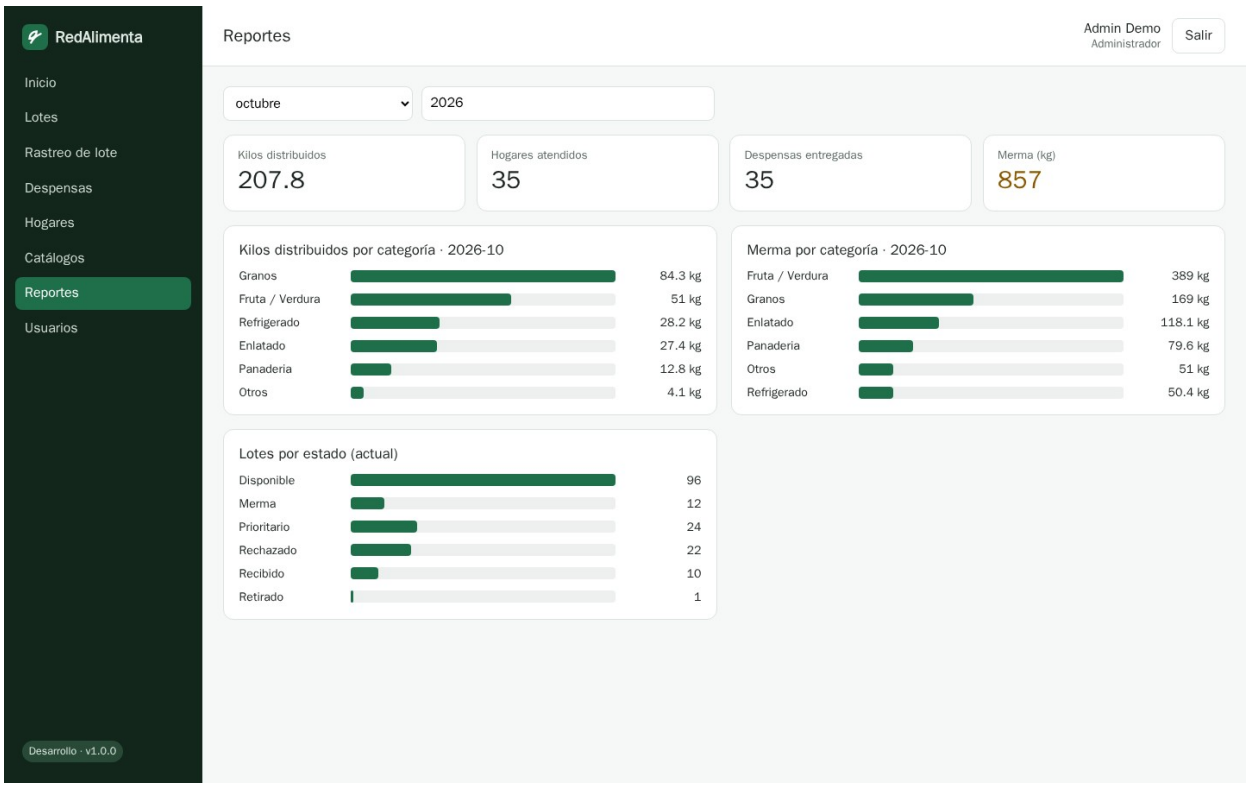
Validación en cliente al registrar un hogar



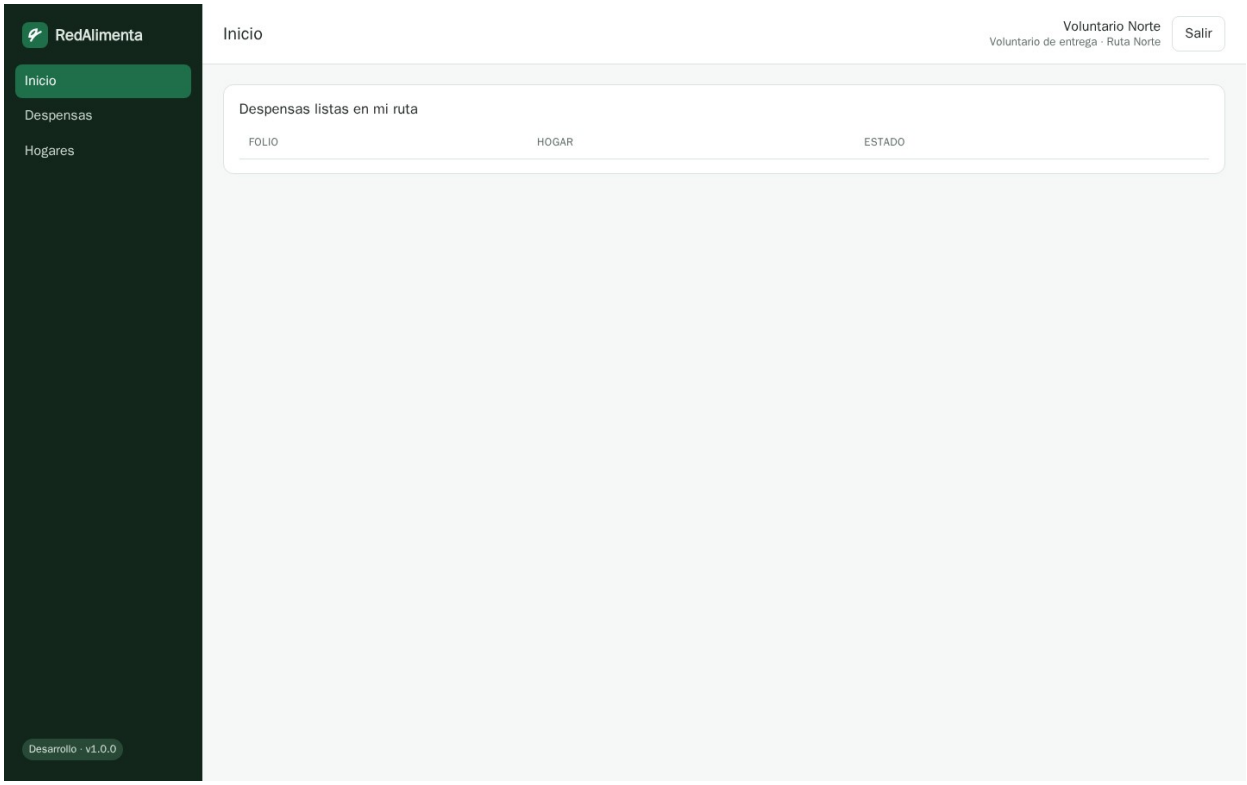
Inspección con checklist propio del tipo de producto



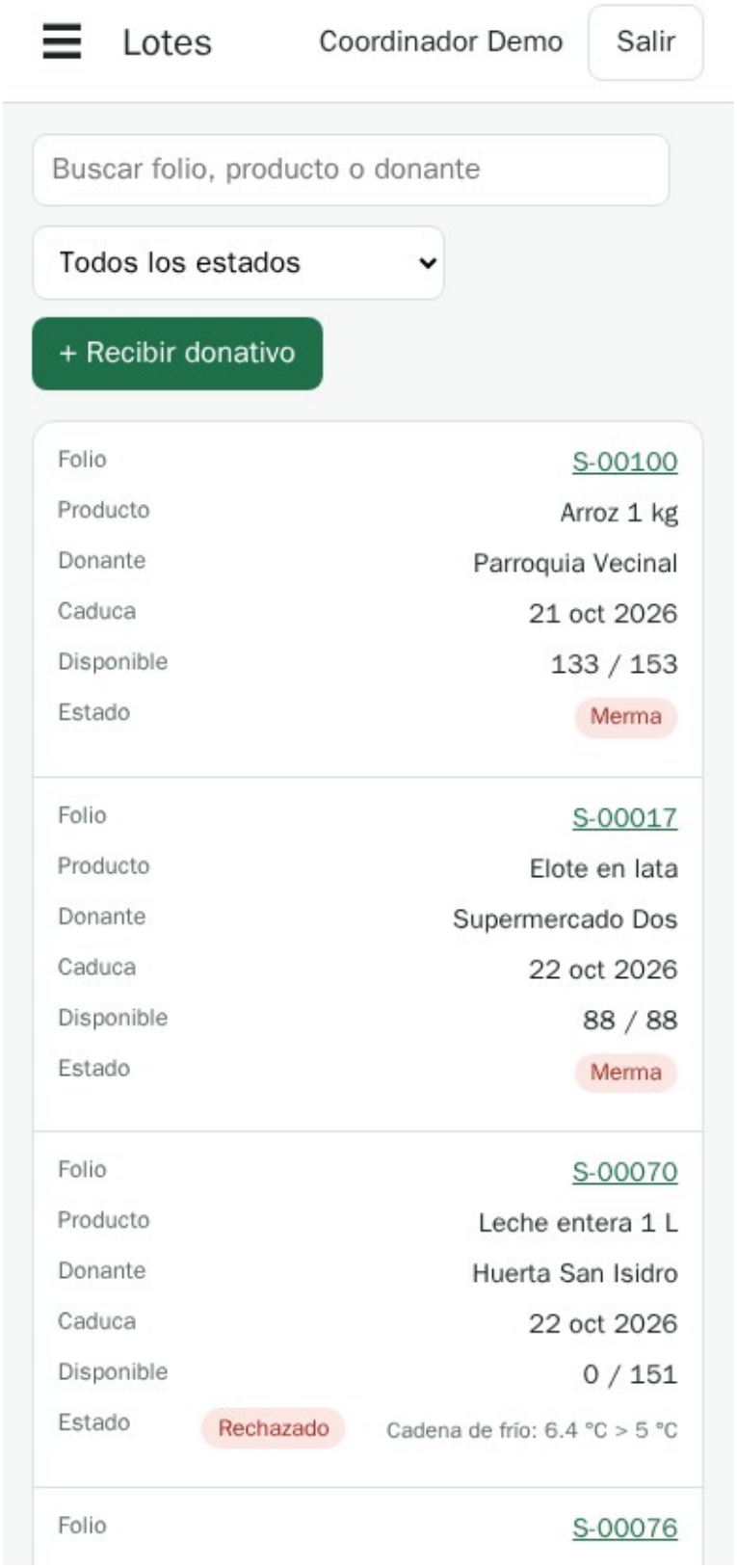
Detalle de despensa con su flujo de estados



Reporte mensual: kilos distribuidos, merma y lotes por estado



Vista del voluntario: solo despensas de su ruta



Interfaz adaptable en móvil

Barreras encontradas

Barrera	Impacto	Cómo se resolvió o mitigó
Sin sudo en el equipo y sin hipervisor accesible (sin libvirt/LXD)	No fue posible crear VMs (kubeadm) ni modificar `/etc/hosts` o el firewall del host	Clústeres k3s en contenedores (k3d); DNS comodín sslip.io; firewall iptables dentro de cada nodo

Barrera	Impacto	Cómo se resolvió o mitigó
Límite fs.inotify.max_user_instances=128 del host	Pods con "too many open files" (frr-k8s de MetalLB; 694 líneas failed to create fsnotify watcher en 25 min en Loki)	Se desactivó FRR (solo L2). **Resuelto** con autorización del usuario: /etc/sysctl.d/99-kubernetes-k3d.conf (1024/524288); 0 errores después (SEC-03)
Longhorn requiere open-iscsi en los nodos	No se pudo usar el almacenamiento replicado de la propuesta	local-path en prod; limitación demostrada en el drain (E11); CSI replicado como ampliación
ingress-nginx retirado por la comunidad	Opción de entrada descartada	Traefik + Gateway API
Hallazgos HIGH/CRITICAL de Trivy en las imágenes base	Build 1.0.0 rechazado	Tratamiento: eliminar pip/setuptools, apk upgrade, quitar curl
topologySpreadConstraints contaba el control plane con taint como dominio vacío	El HPA pidió 6 réplicas y 4 quedaron en Pending	nodeTaintsPolicy: Honor; experimento repetido (E06)
node_memory_MemAvailable_bytes ausente en nodos k3d (meminfo simulado)	Panel y alerta de memoria de nodo sin datos	Memoria de Pods / asignable (cAdvisor + kube-state-metrics)
No hay herramientas de navegador conectadas a la sesión	Sin control directo de Chrome	Capturas con Chromium headless (Playwright 1.64.1) en Docker

Limitaciones

- Ambos clústeres comparten el mismo host físico. El aislamiento es por contenedores, redes Docker, contextos y credenciales; no hay aislamiento de hardware ni de kernel.
- Longhorn no fue viable (sin iSCSI en los nodos k3d). Se usa local-path, así que la base de datos de un worker en mantenimiento no se reubica.
- La IP de MetalLB solo es alcanzable desde el host (la red Docker no está puenteada a la LAN).
- La CA es de laboratorio; un navegador externo necesita importar secrets/ca-*.crt.
- La alta disponibilidad del plano de control y de las bases de datos está diseñada, pero no implementada (ampliación).

Conclusiones

- Los dos ambientes son reproducibles y declarativos: los clústeres se recrean con scripts y los overlays de Kustomize solo difieren en configuración. La promoción mueve el **mismo digest** de dev a prod; la reconstrucción de dev tomó 7 min.
- El patrón outbox resolvió la consistencia entre motores sin suponer atomicidad. Con MongoDB caído, la operación del almacén no se detuvo y la reconciliación no produjo duplicados, incluso forzando la reaplicación.
- Las pruebas encontraron cuatro defectos reales que una revisión estática no habría mostrado: la distribución que contaba el control plane, Traefik como punto único, la falta de preStop en rolling updates y una métrica del outbox que no reflejaba el atraso. Todos se corrigieron y se volvieron a medir.
- El HPA mejora la latencia de cola (p95 −34 %), pero no el throughput con un generador de lazo cerrado. El siguiente cuello de botella es PostgreSQL. Escalar Pods no es lineal ni sustituye la capacidad de los nodos.
- Las limitaciones principales son físicas: un solo host y almacenamiento local. Las ampliaciones prioritarias son nodos o VMs reales, un CSI replicado y la replicación de PostgreSQL (CloudNativePG) y MongoDB (replica set).

Referencias

- Kubernetes — Production environment, HPA, Network Policies, Security Checklist, Application Security Checklist, Pod Security Standards, Topology Spread Constraints (nodeTaintsPolicy). kubernetes.io/docs, consultadas el 31-oct-2026 (v1.37).

- K3s documentation (secrets-encryption, flags del servidor) y k3d v5.9.0 (k3d.io/v1alpha5). docs.k3s.io, k3d.io, consultadas el 31-oct-2026.
- Calico v3.33 — instalación con manifiesto, VXLAN. docs.tigera.io.
- MetalLB 0.16 — modo L2. metallb.io.
- Traefik v3.7 — proveedor Kubernetes Gateway API; Gateway API v1. doc.traefik.io, gateway-api.sigs.k8s.io.
- cert-manager v1.21 — CA Issuer. cert-manager.io.
- PostgreSQL 18 — pg_dump/pg_restore, SSL, pg_hba. MongoDB 9 — TLS, roles integrados, mongodump.
- kube-prometheus-stack 92.17, Grafana Loki 3.6, Grafana Alloy 1.21, k6 1.8.2, Trivy 0.76.1.
- Herramientas de terceros usadas con atribución: FastAPI, SQLAlchemy, Alembic, PyMongo, argon2-cffi, PyJWT, React, Vite, Nginx.

Anexo · Índice de evidencias

Archivo	Contenido
docs/evidencias/E01-contextos-nodos.txt	Contextos, nodos, capacidad asignable y componentes
docs/evidencias/E02-independencia.txt	URLs, CA, datos, credenciales y volúmenes distintos
docs/evidencias/E03-pruebas-funcionales-*.txt	22 pruebas en dev y prod
docs/evidencias/E04-ambos-motores.txt	event_id trazable PostgreSQL → MongoDB, explain, validación
docs/evidencias/E07-balanceo-prod.txt	Reparto de solicitudes entre réplicas
docs/evidencias/E08-*.txt	NetworkPolicy, firewall y reglas exportadas
docs/evidencias/E09-rbac-y-bd.txt	RBAC con identidades reales y permisos de BD
docs/evidencias/E13-*.txt	Respaldos y restauración aislada
docs/evidencias/carga/	Scripts, series CSV, resúmenes k6 y gráficas
docs/evidencias/fallos/	Sondas y líneas de tiempo de fallos, drain y rollback
docs/evidencias/ci/	Builds, digests, reportes Trivy y kubeconform